

timer is usually a precisely machined quartz crystal. When kept under tension, quartz crystals oscillate at a well-defined frequency that depends on the kind of crystal, how it is cut, and the amount of tension. Associated with each crystal are two registers, a **counter** and a **holding register**. Each oscillation of the crystal decrements the counter by one. When the counter gets to zero, an interrupt is generated and the counter is reloaded from the holding register. In this way, it is possible to program a timer to generate an interrupt 60 times a second, or at any other desired frequency. Each interrupt is called one **clock tick**.

When the system is booted initially, it usually asks the operator to enter the date and time, which is then converted to the number of ticks after some known starting date and stored in memory. At every clock tick, the interrupt service procedure adds one to the time stored in memory. In this way, the (software) clock is kept up to date.

With a single computer and a single clock, it does not matter much if this clock is off by a small amount. Since all processes on the machine use the same clock, they will still be internally consistent. For example, if the file *input.c* has time 2151 and file *input.o* has time 2150, *make* will recompile the source file, even if the clock is off by 2 and the true times are 2153 and 2152, respectively. All that really matters are the relative times.

As soon as multiple CPUs are introduced, each with its own clock, the situation changes. Although the frequency at which a crystal oscillator runs is usually fairly stable, it is impossible to guarantee that the crystals in different computers all run at exactly the same frequency. In practice, when a system has n computers, all n crystals will run at slightly different rates, causing the (software) clocks gradually to get out of sync and give different values when read out. This difference in time values is called **clock skew**. As a consequence of this clock skew, programs that expect the time associated with a file, object, process, or message to be correct and independent of the machine on which it was generated (i.e., which clock it used) can fail, as we saw in the *make* example above.

This brings us back to our original question, whether it is possible to synchronize all the clocks to produce a single, unambiguous time standard. In a classic paper, Lamport (1978) showed that clock synchronization is possible and presented an algorithm for achieving it. He extended his work in (Lamport, 1990).

Lamport pointed out that clock synchronization need not be absolute. If two processes do not interact, it is not necessary that their clocks be synchronized because the lack of synchronization would not be observable and thus could not cause problems. Furthermore, he pointed out that what usually matters is not that all processes agree on exactly what time it is, but rather, that they agree on the order in which events occur. In the *make* example above, what counts is whether *input.c* is older or newer than *input.o*, not their absolute creation times.

For many purposes, it is sufficient that all machines agree on the same time. It is not essential that this time also agree with the real time as announced on the radio every hour. For running *make*, for example, it is adequate that all machines agree that it is 10:00, even if it is really 10:02. Thus for a certain class of algorithms, it is the internal consistency of the clocks that matters, not whether they are particularly close to the real time. For these algorithms, it is conventional to speak of the clocks as **logical clocks**.

When the additional constraint is present that the clocks must not only be the same, but also must not deviate from the real time by more than a certain amount, the clocks are called **physical clocks**. In this section we will discuss Lamport's algorithm, which synchronizes logical clocks. In the following sections we will introduce the concept of physical time and show how physical clocks can be synchronized.

To synchronize logical clocks, Lamport defined a relation called **happens-before**. The expression $a \rightarrow b$ is read " a happens before b " and means that all processes agree that first event a occurs, then afterward, event b occurs. The happens-before relation can be observed directly in two situations:

1. If a and b are events in the same process, and a occurs before b , then $a \rightarrow b$ is true.
2. If a is the event of a message being sent by one process, and b is the event of the message being received by another process, then $a \rightarrow b$ is also true. A message cannot be received before it is sent, or even at the same time it is sent, since it takes a finite amount of time to arrive.

Happens-before is a transitive relation, so if $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$. If two events, x and y , happen in different processes that do not exchange messages (not even indirectly via third parties), then $x \rightarrow y$ is not true, but neither is $y \rightarrow x$. These events are said to be **concurrent**, which simply means that nothing can be said (or need be said) about when they happened or which is first.

What we need is a way of measuring time such that for every event, a , we can assign it a time value $C(a)$ on which all processes agree. These time values must have the property that if $a \rightarrow b$, then $C(a) < C(b)$. To rephrase the conditions we stated earlier, if a and b are two events within the same process and a occurs before b , then $C(a) < C(b)$. Similarly, if a is the sending of a message by one process and b is the reception of that message by another process, then $C(a)$ and $C(b)$ must be assigned in such a way that everyone agrees on the values of $C(a)$ and $C(b)$ with $C(a) < C(b)$. In addition, the clock time, C , must always go forward (increasing), never backward (decreasing). Corrections to time can be made by adding a positive value, never by subtracting one.

Now let us look at the algorithm Lamport proposed for assigning times to

events. Consider the three processes depicted in Fig. 3-2(a). The processes run on different machines, each with its own clock, running at its own speed. As can be seen from the figure, when the clock has ticked 6 times in process 0, it has ticked 8 times in process 1 and 10 times in process 2. Each clock runs at a constant rate, but the rates are different due to differences in the crystals.

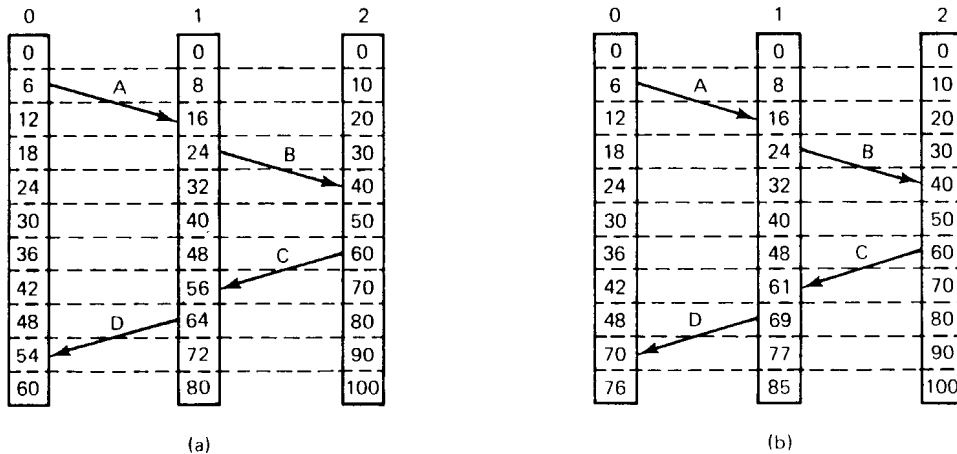


Fig. 3-2. (a) Three processes, each with its own clock. The clocks run at different rates. (b) Lamport's algorithm corrects the clocks.

At time 6, process 0 sends message *A* to process 1. How long this message takes to arrive depends on whose clock you believe. In any event, the clock in process 1 reads 16 when it arrives. If the message carries the starting time, 6, in it, process 1 will conclude that it took 10 ticks to make the journey. This value is certainly possible. According to this reasoning, message *B* from 1 to 2 takes 16 ticks, again a plausible value.

Now comes the fun part. Message *C* from 2 to 1 leaves at 60 and arrives at 56. Similarly, message *D* from 1 to 0 leaves at 64 and arrives at 54. These values are clearly impossible. It is this situation that must be prevented.

Lamport's solution follows directly from the happened-before relation. Since *C* left at 60, it must arrive at 61 or later. Therefore, each message carries the sending time, according to the sender's clock. When a message arrives and the receiver's clock shows a value prior to the time the message was sent, the receiver fast forwards its clock to be one more than the sending time. In Fig. 3-2(b) we see that *C* now arrives at 61. Similarly, *D* arrives at 70.

With one small addition, this algorithm meets our requirements for global time. The addition is that between every two events, the clock must tick at least once. If a process sends or receives two messages in quick succession, it must advance its clock by (at least) one tick in between them.

In some situations, an additional requirement is desirable: no two events ever occur at exactly the same time. To achieve this goal, we can attach the number of the process in which the event occurs to the low-order end of the time, separated by a decimal point. Thus if events happen in processes 1 and 2, both with time 40, the former becomes 40.1 and the latter becomes 40.2.

Using this method, we now have a way to assign time to all events in a distributed system subject to the following conditions:

1. If a happens before b in the same process, $C(a) < C(b)$.
2. If a and b represent the sending and receiving of a message, $C(a) < C(b)$.
3. For all events a and b , $C(a) \neq C(b)$.

This algorithm gives us a way to provide a total ordering of all events in the system. Many other distributed algorithms need such an ordering to avoid ambiguities, so the algorithm is widely cited in the literature.

3.1.2. Physical Clocks

Although Lamport's algorithm gives an unambiguous event ordering, the time values assigned to events are not necessarily close to the actual times at which they occur. In some systems (e.g., real-time systems), the actual clock time is important. For these systems external physical clocks are required. For reasons of efficiency and redundancy, multiple physical clocks are generally considered desirable, which yields two problems: (1) How do we synchronize them with real-world clocks, and (2) How do we synchronize the clocks with each other?

Before answering these questions, let us digress slightly to see how time is actually measured. It is not nearly as simple as one might think, especially when high accuracy is required. Since the invention of mechanical clocks in the 17th century, time has been measured astronomically. Every day, the sun appears to rise on the eastern horizon, climbs to a maximum height in the sky, and sinks in the west. The event of the sun's reaching its highest apparent point in the sky is called the **transit of the sun**. This event occurs at about noon each day. The interval between two consecutive transits of the sun is called the **solar day**. Since there are 24 hours in a day, each containing 3600 seconds, the **solar second** is defined as exactly 1/86400th of a solar day. The geometry of the mean solar day calculation is shown in Fig. 3-3.

In the 1940s, it was established that the period of the earth's rotation is not constant. The earth is slowing down due to tidal friction and atmospheric drag. Based on studies of growth patterns in ancient coral, geologists now believe that

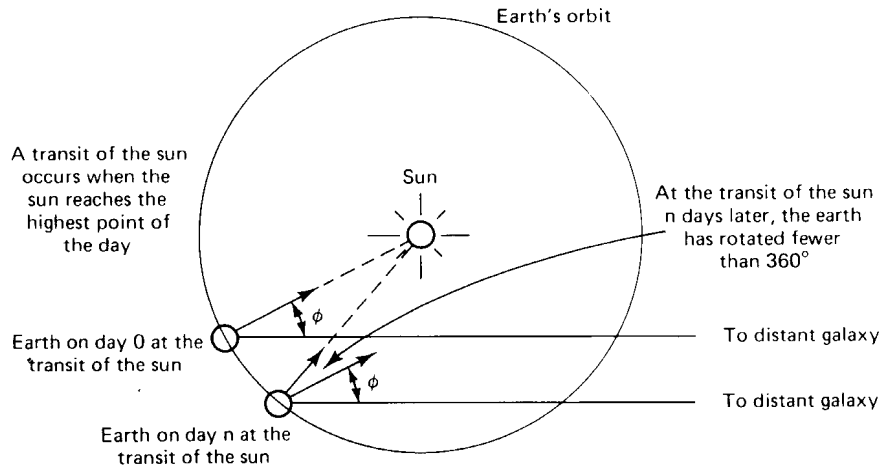


Fig. 3-3. Computation of the mean solar day.

300 million years ago there were about 400 days per year. The length of the year, that is, the time for one trip around the sun, is not thought to have changed; the day has simply become longer. In addition to this long-term trend, short-term variations in the length of the day also occur, probably caused by turbulence deep in the earth's core of molten iron. These revelations led astronomers to compute the length of the day by measuring a large number of days and taking the average before dividing by 86,400. The resulting quantity was called the **mean solar second**.

With the invention of the atomic clock in 1948, it became possible to measure time much more accurately, and independent of the wiggling and wobbling of the earth, by counting transitions of the cesium 133 atom. The physicists took over the job of timekeeping from the astronomers, and defined the second to be the time it takes the cesium 133 atom to make exactly 9,192,631,770 transitions. The choice of 9,192,631,770 was made to make the atomic second equal to the mean solar second in the year of its introduction. Currently, about 50 laboratories around the world have cesium 133 clocks. Periodically, each laboratory tells the Bureau International de l'Heure (BIH) in Paris how many times its clock has ticked. The BIH averages these to produce **International Atomic Time**, which is abbreviated **TAI**. Thus TAI is just the mean number of ticks of the cesium 133 clocks since midnight on Jan. 1, 1958 (the beginning of time) divided by 9,192,631,770.

Although TAI is highly stable and available to anyone who wants to go to the trouble of buying a cesium clock, there is a serious problem with it; 86,400 TAI seconds is now about 3 msec less than a mean solar day (because the mean

solar day is getting longer all the time). Using TAI for keeping time would mean that over the course of the years, noon would get earlier and earlier, until it would eventually occur in the wee hours of the morning. People might notice this and we could have the same kind of situation as occurred in 1582 when Pope Gregory XIII decreed that 10 days be omitted from the calendar. This event caused riots in the streets because landlords demanded a full month's rent and bankers a full month's interest, while employers refused to pay workers for the 10 days they did not work, to mention only a few of the conflicts. The Protestant countries, as a matter of principle, refused to have anything to do with papal decrees and did not accept the Gregorian calendar for 170 years.

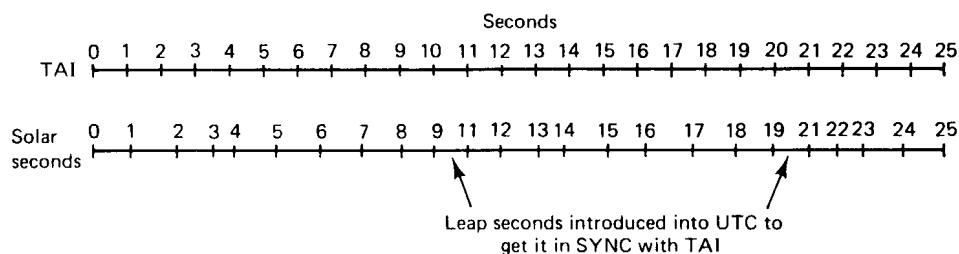


Fig. 3-4. TAI seconds are of constant length, unlike solar seconds. Leap seconds are introduced when necessary to keep in phase with the sun.

BIH solves the problem by introducing **leap seconds** whenever the discrepancy between TAI and solar time grows to 800 msec. The use of leap seconds is illustrated in Fig. 3-4. This correction gives rise to a time system based on constant TAI seconds but which stays in phase with the apparent motion of the sun. It is called **Universal Coordinated Time**, but is abbreviated as **UTC**. UTC is the basis of all modern civil timekeeping. It has essentially replaced the old standard, Greenwich Mean Time, which is astronomical time.

Most electric power companies base the timing of their 60-Hz or 50-Hz clocks on UTC, so when BIH announces a leap second, the power companies raise their frequency to 61 Hz or 51 Hz for 60 or 50 sec, to advance all the clocks in their distribution area. Since 1 sec is a noticeable interval for a computer, an operating system that needs to keep accurate time over a period of years must have special software to account for leap seconds as they are announced (unless they use the power line for time, which is usually too crude). The total number of leap seconds introduced into UTC so far is about 30.

To provide UTC to people who need precise time, the National Institute of Standard Time (NIST) operates a shortwave radio station with call letters WWV from Fort Collins, Colorado. WWV broadcasts a short pulse at the start of each UTC second. The accuracy of WWV itself is about ± 1 msec, but due to random atmospheric fluctuations that can affect the length of the signal path, in practice

the accuracy is no better than ± 10 msec. In England, the station MSF, operating from Rugby, Warwickshire, provides a similar service, as do stations in several other countries.

Several earth satellites also offer a UTC service. The Geostationary Environment Operational Satellite can provide UTC accurately to 0.5 msec, and some other satellites do even better.

Using either shortwave radio or satellite services requires an accurate knowledge of the relative position of the sender and receiver, in order to compensate for the signal propagation delay. Radio receivers for WWV, GEOS, and the other UTC sources are commercially available. The cost varies from a few thousand dollars each to tens of thousands of dollars each, being more for the better sources. UTC can also be obtained more cheaply, but less accurately, by telephone from NIST in Fort Collins, but here too, a correction must be made for the signal path and modem speed. This correction introduces some uncertainty, making it difficult to obtain the time with extremely high accuracy.

3.1.3. Clock Synchronization Algorithms

If one machine has a WWV receiver, the goal becomes keeping all the other machines synchronized to it. If no machines have WWV receivers, each machine keeps track of its own time, and the goal is to keep all the machines together as well as possible. Many algorithms have been proposed for doing this synchronization (e.g., Cristian, 1989; Drummond and Babaoglu, 1993; and Kopetz and Ochsenreiter, 1987). A survey is given in (Ramanathan et al., 1990b).

All the algorithms have the same underlying model of the system, which we will now describe. Each machine is assumed to have a timer that causes an interrupt H times a second. When this timer goes off, the interrupt handler adds 1 to a software clock that keeps track of the number of ticks (interrupts) since some agreed-upon time in the past. Let us call the value of this clock C . More specifically, when the UTC time is t , the value of the clock on machine p is $C_p(t)$. In a perfect world, we would have $C_p(t) = t$ for all p and all t . In other words, dC/dt ideally should be 1.

Real timers do not interrupt exactly H times a second. Theoretically, a timer with $H = 60$ should generate 216,000 ticks per hour. In practice, the relative error obtainable with modern timer chips is about 10^{-5} , meaning that a particular machine can get a value in the range 215,998 to 216,002 ticks per hour. More precisely, if there exists some constant ρ such that

$$1 - \rho \leq \frac{dC}{dt} \leq 1 + \rho$$

the timer can be said to be working within its specification. The constant ρ is

specified by the manufacturer and is known as the **maximum drift rate**. Slow, perfect, and fast clocks are shown in Fig. 3-5.

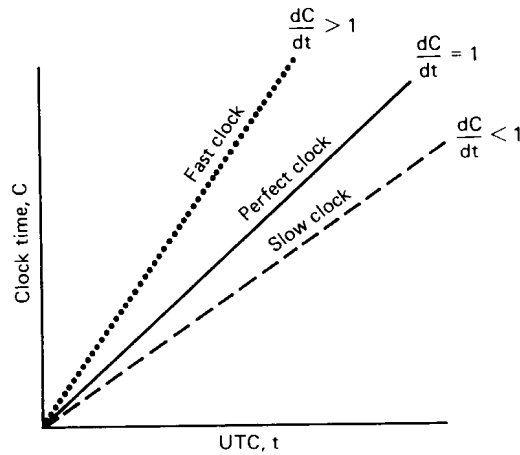


Fig. 3-5. Not all clocks tick precisely at the correct rate.

If two clocks are drifting from UTC in the opposite direction, at a time Δt after they were synchronized, they may be as much as $2\rho \Delta t$ apart. If the operating system designers want to guarantee that no two clocks ever differ by more than δ , clocks must be resynchronized (in software) at least every $\delta/2\rho$ seconds. The various algorithms differ in precisely how this resynchronization is done.

Cristian's Algorithm

Let us start with an algorithm that is well suited to systems in which one machine has a WWV receiver and the goal is to have all the other machines stay synchronized with it. Let us call the machine with the WWV receiver a **time server**. Our algorithm is based on the work of Cristian (1989) and prior work. Periodically, certainly no more than every $\delta/2\rho$ seconds, each machine sends a message to the time server asking it for the current time. That machine responds as fast as it can with a message containing its current time, C_{UTC} , as shown in Fig. 3-6.

As a first approximation, when the sender gets the reply, it can just set its clock to C_{UTC} . However, this algorithm has two problems, one major and one minor. The major problem is that time must never run backward. If the sender's clock is fast, C_{UTC} will be smaller than the sender's current value of C . Just taking over C_{UTC} could cause serious problems, such as an object file compiled just after the clock change having a time earlier than the source which was modified just before the clock change.

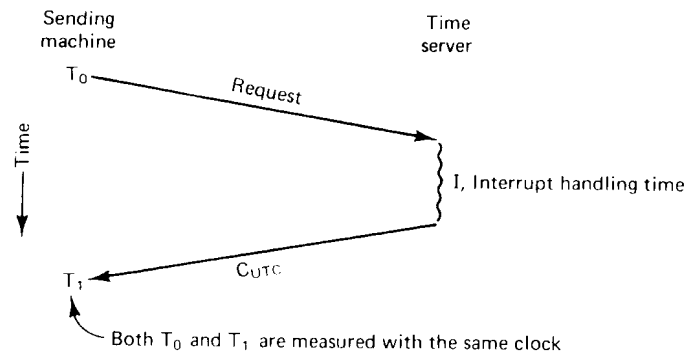


Fig. 3-6. Getting the current time from a time server.

Such a change must be introduced gradually. One way is as follows. Suppose that the timer is set to generate 100 interrupts per second. Normally, each interrupt would add 10 msec to the time. When slowing down, the interrupt routine adds only 9 msec each time, until the correction has been made. Similarly, the clock can be advanced gradually by adding 11 msec at each interrupt instead of jumping it forward all at once.

The minor problem is that it takes a nonzero amount of time for the time server's reply to get back to the sender. Worse yet, this delay may be large and vary with the network load. Cristian's way of dealing with it is to attempt to measure it. It is simple enough for the sender to record accurately the interval between sending the request to the time server and the arrival of the reply. Both the starting time, T_0 , and the ending time, T_1 , are measured using the same clock, so the interval will be relatively accurate, even if the sender's clock is off from UTC by a substantial amount.

In the absence of any other information, the best estimate of the message propagation time is $(T_1 - T_0)/2$. When the reply comes in, the value in the message can be increased by this amount to give an estimate of the server's current time. If the theoretical minimum propagation time is known, other properties of the time estimate can be calculated.

This estimate can be improved if it is known approximately how long it takes the time server to handle the interrupt and process the incoming message. Let us call the interrupt handling time I . Then the amount of the interval from T_0 to T_1 that was devoted to message propagation is $T_1 - T_0 - I$, so the best estimate of the one-way propagation time is half this. Systems do exist in which messages from A to B systematically take a different route than messages from B to A , and thus have a different propagation time, but we will not consider such systems here.

To improve the accuracy, Cristian suggested making not one measurement,

but a series of them. Any measurements in which $T_1 - T_0$ exceeds some threshold value are discarded as being victims of network congestion and thus unreliable. The estimates derived from the remaining probes can then be averaged to get a better value. Alternatively, the message that came back fastest can be taken to be the most accurate since it presumably encountered the least traffic underway and thus is the most representative of the pure propagation time.

The Berkeley Algorithm

In Cristian's algorithm, the time server is passive. Other machines ask it for the time periodically. All it does is respond to their queries. In Berkeley UNIX, exactly the opposite approach is taken (Gusella and Zatti, 1989). Here the time server (actually, a time daemon) is active, polling every machine periodically to ask what time it is there. Based on the answers, it computes an average time and tells all the other machines to advance their clocks to the new time or slow their clocks down until some specified reduction has been achieved. This method is suitable for a system in which no machine has a WWV receiver. The time daemon's time must be set manually by the operator periodically. The method is illustrated in Fig. 3-7.

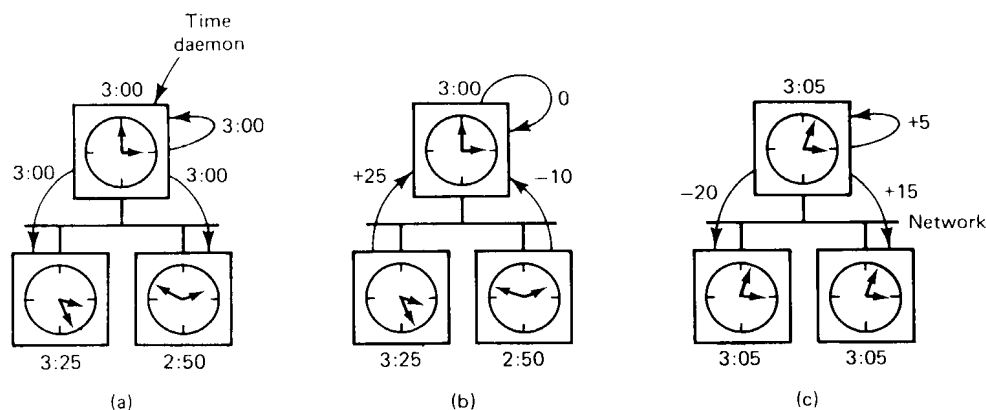


Fig. 3-7. (a) The time daemon asks all the other machines for their clock values. (b) The machines answer. (c) The time daemon tells everyone how to adjust their clock.

In Fig. 3-7(a), at 3:00, the time daemon tells the other machines its time and asks for theirs. In Fig. 3-7(b), they respond with how far ahead or behind the time daemon they are. Armed with these numbers, the time daemon computes the average and tells each machine how to adjust its clock [see Fig. 3-7(c)].

Averaging Algorithms

Both of the methods described above are highly centralized, with the usual disadvantages. Decentralized algorithms are also known. One class of decentralized clock synchronization algorithms works by dividing time into fixed-length resynchronization intervals. The i th interval starts at $T_0 + iR$ and runs until $T_0 + (i + 1)R$, where T_0 is an agreed upon moment in the past, and R is a system parameter. At the beginning of each interval, every machine broadcasts the current time according to its clock. Because the clocks on different machines do not run at exactly the same speed, these broadcasts will not happen precisely simultaneously.

After a machine broadcasts its time, it starts a local timer to collect all other broadcasts that arrive during some interval S . When all the broadcasts arrive, an algorithm is run to compute a new time from them. The simplest algorithm is just to average the values from all the other machines. A slight variation on this theme is first to discard the m highest and m lowest values, and average the rest. Discarding the extreme values can be regarded as self defense against up to m faulty clocks sending out nonsense.

Another variation is to try to correct each message by adding to it an estimate of the propagation time from the source. This estimate can be made from the known topology of the network, or by timing how long it takes for probe messages to be echoed.

Additional clock synchronization algorithms are discussed in the literature (e.g., Lundelius-Welch and Lynch, 1988; Ramanathan et al., 1990a; and Srikanth and Toueg, 1987).

Multiple External Time Sources

For systems in which extremely accurate synchronization with UTC is required, it is possible to equip the system with multiple receivers for WWV, GEOS, or other UTC sources. However, due to inherent inaccuracy in the time source itself as well as fluctuations in the signal path, the best the operating system can do is establish a range (time interval) in which UTC falls. In general, the various time sources will produce different ranges, which requires the machines attached to them to come to agreement.

To reach this agreement, each processor with a UTC source can broadcast its range periodically, say, at the precise start of each UTC minute. None of the processors will get the time packets instantaneously. Worse yet, the delay between transmission and reception depends on the cable distance and number of gateways that the packets have to traverse, which is different for each (UTC source, processor) pair. Other factors can also play a role, such as delays due to collisions when multiple machines try to transmit on an Ethernet at the same

instant. Furthermore, if a processor is busy handling a previous packet, it may not even look at the time packet for a considerable number of milliseconds, introducing additional uncertainty into the time. In Chap. 10 we will examine how clocks are synchronized in OSF's DCE.

3.1.4. Use of Synchronized Clocks

Only quite recently has the necessary hardware and software for synchronizing clocks on a wide scale (e.g., over the entire Internet) become easily available. With this new technology, it is possible to keep millions of clocks synchronized to within a few milliseconds of UTC. New algorithms that utilize synchronized clocks are just starting to appear. Below we summarize two of the examples discussed by Liskov (1993).

At-Most-Once Message Delivery

Our first example concerns how to enforce at-most-once message delivery to a server, even in the face of crashes. The traditional approach is for each message to bear a unique message number, and have each server store all the numbers of the messages it has seen so it can detect new messages from retransmissions. The problem with this algorithm is that if a server crashes and reboots, it loses its table of message numbers. Also, for how long should message numbers be saved?

Using time, the algorithm can be modified as follows. Now, every message carries a connection identifier (chosen by the sender) and a timestamp. For each connection, the server records in a table the most recent timestamp it has seen. If any incoming message for a connection is lower than the timestamp stored for that connection, the message is rejected as a duplicate.

To make it possible to remove old timestamps, each server continuously maintains a global variable

$$G = \text{CurrentTime} - \text{MaxLifetime} - \text{MaxClockSkew}$$

where *MaxLifetime* is the maximum time a message can live and *MaxClockSkew* is how far from UTC the clock might be at worst. Any timestamp older than *G* can safely be removed from the table because all messages that old have died out already. If an incoming message has an unknown connection identifier, it is accepted if its timestamp is more recent than *G* and rejected if its timestamp is older than *G* because anything that old surely is a duplicate. In effect, *G* is a summary of the message numbers of all old messages. Every ΔT , the current time is written to disk.

When a server crashes and then reboots, it reloads *G* from the time stored on

disk and increments it by the update period, ΔT . Any incoming message with a timestamp older than G is rejected as a duplicate. As a consequence, every message that might have been accepted before the crash is rejected. Some new messages may be incorrectly rejected, but under all conditions the algorithm maintains at-most-once semantics.

Clock-Based Cache Consistency

Our second example concerns cache consistency in a distributed file system. For performance reasons, it is desirable for clients to be able to cache files locally. However, caching introduces potential inconsistency if two clients modify the same file at the same time. The usual solution is to distinguish between caching a file for reading and caching a file for writing. The disadvantage of this scheme is that if a client has a file cached for reading, before another client can get a copy for writing, the server has to first ask the reading client to invalidate its copy, even if the copy was made hours ago. This extra overhead can be eliminated using synchronized clocks.

The basic idea is that when a client wants a file, it is given a **lease** on it that specifies how long the copy is valid (Gray and Cheriton, 1989). When the lease is about to expire, the client can ask for it to be renewed. If a lease expires, the cached copy may no longer be used. In this way when a client needs to read a file once, it can ask for it. When the lease expires, it just times out; there is no need to explicitly send a message telling the server that it has been purged from the cache.

If a lease has expired and the file (still cached) is needed again shortly thereafter, the client can ask the server if the copy it has (identified by a timestamp) is still the current one. If so, a new lease is generated, but the file need not be retransmitted.

If one or more clients have a file cached for reading and then another client wants to write on the file, the server has to ask the readers to prematurely terminate their leases. If one or more of them has crashed, the server can just wait until the dead server's lease times out. In the traditional algorithm, where permission-to-cache must be returned explicitly from the client to the server, a problem occurs if the server asks the client or clients to return the file (i.e., discard it from its cache) and there is no response. The server cannot tell if the client is dead or merely slow. With the timer-based algorithm, the server can just wait and let the lease expire.

In addition to these two algorithms, Liskov (1993) also describes how synchronized clocks can be used to time out tickets used in distributed system authentication, and handle commitment in atomic transactions. As timer synchronization gets better, no doubt new applications for it will be found.

3.2. MUTUAL EXCLUSION

Systems involving multiple processes are often most easily programmed using critical regions. When a process has to read or update certain shared data structures, it first enters a critical region to achieve mutual exclusion and ensure that no other process will use the shared data structures at the same time. In single-processor systems, critical regions are protected using semaphores, monitors, and similar constructs. We will now look at a few examples of how critical regions and mutual exclusion can be implemented in distributed systems. For a taxonomy and bibliography of other methods, see (Raynal, 1991). Other work is discussed in (Agrawal and El Abbadi, 1991; Chandy et al., 1983; and Sanders, 1987).

3.2.1. A Centralized Algorithm

The most straightforward way to achieve mutual exclusion in a distributed system is to simulate how it is done in a one-processor system. One process is elected as the coordinator (e.g., the one running on the machine with the highest network address). Whenever a process wants to enter a critical region, it sends a request message to the coordinator stating which critical region it wants to enter and asking for permission. If no other process is currently in that critical region, the coordinator sends back a reply granting permission, as shown in Fig. 3-8(a). When the reply arrives, the requesting process enters the critical region.

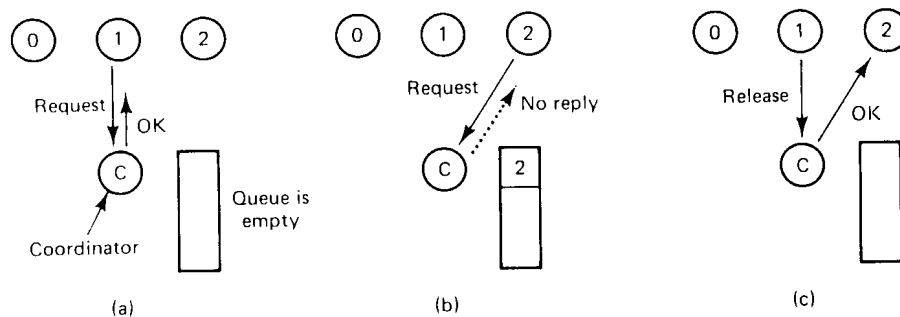


Fig. 3-8. (a) Process 1 asks the coordinator for permission to enter a critical region. Permission is granted. (b) Process 2 then asks permission to enter the same critical region. The coordinator does not reply. (c) When process 1 exits the critical region, it tells the coordinator, which then replies to 2.

Now suppose that another process, 2 in Fig. 3-8(b), asks for permission to enter the same critical region. The coordinator knows that a different process is already in the critical region, so it cannot grant permission. The exact method used to deny permission is system dependent. In Fig. 3-8(b), the coordinator just

refrains from replying, thus blocking process 2, which is waiting for a reply. Alternatively, it could send a reply saying “permission denied.” Either way, it queues the request from 2 for the time being.

When process 1 exits the critical region, it sends a message to the coordinator releasing its exclusive access, as shown in Fig. 3-8(c). The coordinator takes the first item off the queue of deferred requests and sends that process a grant message. If the process was still blocked (i.e., this is the first message to it), it unblocks and enters the critical region. If an explicit message has already been sent denying permission, the process will have to poll for incoming traffic, or block later. Either way, when it sees the grant, it can enter the critical region.

It is easy to see that the algorithm guarantees mutual exclusion: the coordinator only lets one process at a time into each critical region. It is also fair, since requests are granted in the order in which they are received. No process ever waits forever (no starvation). The scheme is easy to implement, too, and requires only three messages per use of a critical region (request, grant, release). It can also be used for more general resource allocation rather than just managing critical regions.

The centralized approach also has shortcomings. The coordinator is a single point of failure, so if it crashes, the entire system may go down. If processes normally block after making a request, they cannot distinguish a dead coordinator from “permission denied” since in both cases no message comes back. In addition, in a large system, a single coordinator can become a performance bottleneck.

3.2.2. A Distributed Algorithm

Having a single point of failure is frequently unacceptable, so researchers have looked for distributed mutual exclusion algorithms. Lamport’s 1978 paper on clock synchronization presented the first one. Ricart and Agrawala (1981) made it more efficient. In this section we will describe their method.

Ricart and Agrawala’s algorithm requires that there be a total ordering of all events in the system. That is, for any pair of events, such as messages, it must be unambiguous which one happened first. Lamport’s algorithm presented in Sec. 3.1.1 is one way to achieve this ordering and can be used to provide time-stamps for distributed mutual exclusion.

The algorithm works as follows. When a process wants to enter a critical region, it builds a message containing the name of the critical region it wants to enter, its process number, and the current time. It then sends the message to all other processes, conceptually including itself. The sending of messages is assumed to be reliable; that is, every message is acknowledged. Reliable group communication if available, can be used instead of individual messages.

When a process receives a request message from another process, the action it takes depends on its state with respect to the critical region named in the message. Three cases have to be distinguished:

1. If the receiver is not in the critical region and does not want to enter it, it sends back an *OK* message to the sender.
2. If the receiver is already in the critical region, it does not reply. Instead, it queues the request.
3. If the receiver wants to enter the critical region but has not yet done so, it compares the timestamp in the incoming message with the one contained in the message that it has sent everyone. The lowest one wins. If the incoming message is lower, the receiver sends back an *OK* message. If its own message has a lower timestamp, the receiver queues the incoming request and sends nothing.

After sending out requests asking permission to enter a critical region, a process sits back and waits until everyone else has given permission. As soon as all the permissions are in, it may enter the critical region. When it exits the critical region, it sends *OK* messages to all processes on its queue and deletes them all from the queue.

Let us try to understand why the algorithm works. If there is no conflict, it clearly works. However, suppose that two processes try to enter the same critical region simultaneously, as shown in Fig. 3-9(a).

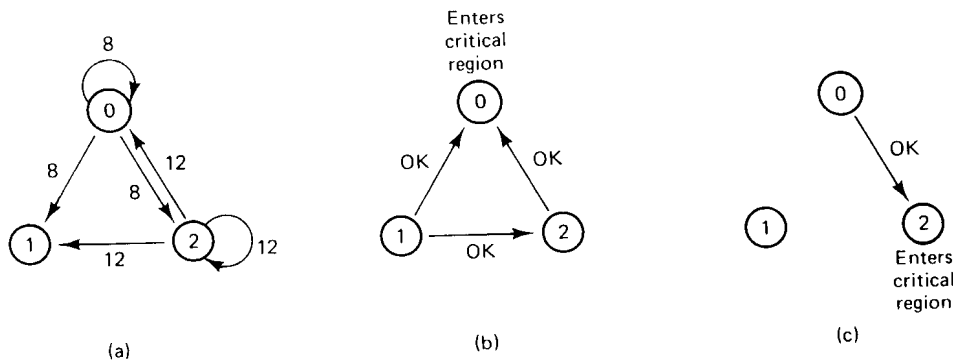


Fig. 3-9. (a) Two processes want to enter the same critical region at the same moment. (b) Process 0 has the lowest timestamp, so it wins. (c) When process 0 is done, it sends an *OK* also, so 2 can now enter the critical region.

Process 0 sends everyone a request with timestamp 8, while at the same time, process 2 sends everyone a request with timestamp 12. Process 1 is not

interested in entering the critical region, so it sends *OK* to both senders. Processes 0 and 2 both see the conflict and compare timestamps. Process 2 sees that it has lost, so it grants permission to 0 by sending *OK*. Process 0 now queues the request from 2 for later processing and enters the critical region, as shown in Fig. 3-9(b). When it is finished, it removes the request from 2 from its queue and sends an *OK* message to process 2, allowing the latter to enter its critical region, as shown in Fig. 3-9(c). The algorithm works because in the case of a conflict, the lowest timestamp wins and everyone agrees on the ordering of the timestamps.

Note that the situation in Fig. 3-9 would have been essentially different if process 2 had sent its message earlier in time so that process 0 had gotten it and granted permission before making its own request. In this case, 2 would have noticed that it itself was in a critical region at the time of the request, and queued it instead of sending a reply.

As with the centralized algorithm discussed above, mutual exclusion is guaranteed without deadlock or starvation. The number of messages required per entry is now $2(n - 1)$, where the total number of processes in the system is n . Best of all, no single point of failure exists.

Unfortunately, the single point of failure has been replaced by n points of failure. If any process crashes, it will fail to respond to requests. This silence will be interpreted (incorrectly) as denial of permission, thus blocking all subsequent attempts by all processes to enter all critical regions. Since the probability of one of the n processes failing is n times as large as a single coordinator failing, we have managed to replace a poor algorithm with one that is n times worse and requires much more network traffic to boot.

The algorithm can be patched up by the same trick that we proposed earlier. When a request comes in, the receiver always sends a reply, either granting or denying permission. Whenever either a request or a reply is lost, the sender times out and keeps trying until either a reply comes back or the sender concludes that the destination is dead. After a request is denied, the sender should block waiting for a subsequent *OK* message.

Another problem with this algorithm is that either a group communication primitive must be used, or each process must maintain the group membership list itself, including processes entering the group, leaving the group, and crashing. The method works best with small groups of processes that never change their group memberships.

Finally, recall that one of the problems with the centralized algorithm is that making it handle all requests can lead to a bottleneck. In the distributed algorithm, *all* processes are involved in *all* decisions concerning entry into critical regions. If one process is unable to handle the load, it is unlikely that forcing everyone to do exactly the same thing in parallel is going to help much.

Various minor improvements are possible to this algorithm. For example,

getting permission from everyone to enter a critical region is really overkill. All that is needed is a method to prevent two processes from entering the critical region at the same time. The algorithm can be modified to allow a process to enter a critical region when it has collected permission from a simple majority of the other processes, rather than from all of them. Of course, in this variation, after a process has granted permission to one process to enter a critical region, it cannot grant the same permission to another process until the first one has released that permission. Other improvements are also possible (e.g., Maekawa et al., 1987).

Nevertheless, this algorithm is slower, more complicated, more expensive, and less robust than the original centralized one. Why bother studying it under these conditions? For one thing, it shows that a distributed algorithm is at least possible, something that was not obvious when we started. Also, by pointing out the shortcomings, we may stimulate future theoreticians to try to produce algorithms that are actually useful. Finally, like eating spinach and learning Latin in high school, some things are said to be good for you in some abstract way.

3.2.3. A Token Ring Algorithm

A completely different approach to achieving mutual exclusion in a distributed system is illustrated in Fig. 3-10. Here we have a bus network, as shown in Fig. 3-10(a), (e.g., Ethernet), with no inherent ordering of the processes. In software, a logical ring is constructed in which each process is assigned a position in the ring, as shown in Fig. 3-10(b). The ring positions may be allocated in numerical order of network addresses or some other means. It does not matter what the ordering is. All that matters is that each process knows who is next in line after itself.

When the ring is initialized, process 0 is given a **token**. The token circulates around the ring. It is passed from process k to process $k + 1$ (modulo the ring size) in point-to-point messages. When a process acquires the token from its neighbor, it checks to see if it is attempting to enter a critical region. If so, the process enters the region, does all the work it needs to, and leaves the region. After it has exited, it passes the token along the ring. It is not permitted to enter a second critical region using the same token.

If a process is handed the token by its neighbor and is not interested in entering a critical region, it just passes it along. As a consequence, when no processes want to enter any critical regions, the token just circulates at high speed around the ring.

The correctness of this algorithm is evident. Only one process has the token at any instant, so only one process can be in a critical region. Since the token circulates among the processes in a well-defined order, starvation cannot occur. Once a process decides it wants to enter a critical region, at worst it will have to

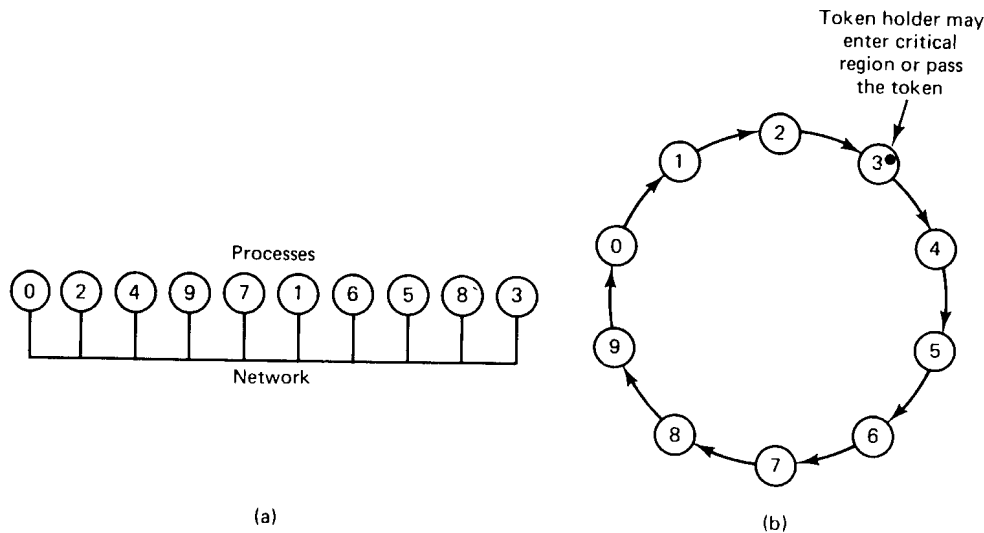


Fig. 3-10. (a) An unordered group of processes on a network. (b) A logical ring constructed in software.

wait for every other process to enter and leave one critical region.

As usual, this algorithm has problems too. If the token is ever lost, it must be regenerated. In fact, detecting that it is lost is difficult, since the amount of time between successive appearances of the token on the network is unbounded. The fact that the token has not been spotted for an hour does not mean that it has been lost; somebody may still be using it.

The algorithm also runs into trouble if a process crashes, but recovery is easier than in the other cases. If we require a process receiving the token to acknowledge receipt, a dead process will be detected when its neighbor tries to give it the token and fails. At that point the dead process can be removed from the group, and the token holder can throw the token over the head of the dead process to the next member down the line, or the one after that, if necessary. Of course, doing so requires that everyone maintains the current ring configuration.

3.2.4. A Comparison of the Three Algorithms

A brief comparison of the three mutual exclusion algorithms we have looked at is instructive. In Fig. 3-11 we have listed the algorithms and three key properties: the number of messages required for a process to enter and exit a critical region, the delay before entry can occur (assuming messages are passed sequentially over a LAN), and some problems associated with each algorithm.

The centralized algorithm is simplest and also most efficient. It requires

Algorithm	Messages per entry/exit	Delay before entry (in message times)	Problems
Centralized	3	2	Coordinator crash
Distributed	$2(n - 1)$	$2(n - 1)$	Crash of any process
Token ring	1 to ∞	0 to $n - 1$	Lost token, process crash

Fig. 3-11. A comparison of three mutual exclusion algorithms.

only three messages to enter and leave a critical region: a request and a grant to enter, and a release to exit. The distributed algorithm requires $n - 1$ request messages, one to each of the other processes, and an additional $n - 1$ grant messages, for a total of $2(n - 1)$. With the token ring algorithm, the number is variable. If every process constantly wants to enter a critical region, then each token pass will result in one entry and exit, for an average of one message per critical region entered. At the other extreme, the token may sometimes circulate for hours without anyone being interested in it. In this case, the number of messages per entry into a critical region is unbounded.

The delay from the moment a process needs to enter a critical region until its actual entry also varies for the three algorithms. When critical regions are short and rarely used, the dominant factor in the delay is the actual mechanism for entering a critical region. When they are long and frequently used, the dominant factor is waiting for everyone else to take their turn. In Fig. 3-11 we show the former case. It takes only two message times to enter a critical region in the centralized case, but $2(n - 1)$ message times in the distributed case, assuming that the network can handle only one message at a time. For the token ring, the time varies from 0 (token just arrived) to $n - 1$ (token just departed).

Finally, all three algorithms suffer badly in the event of crashes. Special measures and additional complexity must be introduced to avoid having a crash bring down the entire system. It is slightly ironic that the distributed algorithms are even more sensitive to crashes than the centralized one. In a fault-tolerant system, none of these would be suitable, but if crashes are very infrequent, they are all acceptable.

3.3. ELECTION ALGORITHMS

Many distributed algorithms require one process to act as coordinator, initiator, sequencer, or otherwise perform some special role. We have already seen several examples, such as the coordinator in the centralized mutual exclusion

algorithm. In general, it does not matter which process takes on this special responsibility, but one of them has to do it. In this section we will look at algorithms for electing a coordinator (using this as a generic name for the special process).

If all processes are exactly the same, with no distinguishing characteristics, there is no way to select one of them to be special. Consequently, we will assume that each process has a unique number, for example its network address (for simplicity, we will assume one process per machine). In general, election algorithms attempt to locate the process with the highest process number and designate it as coordinator. The algorithms differ in the way they do the location.

Furthermore, we also assume that every process knows the process number of every other process. What the processes do not know is which ones are currently up and which ones are currently down. The goal of an election algorithm is to ensure that when an election starts, it concludes with all processes agreeing on who the new coordinator is to be. Various algorithms are known, for example, (Fredrickson and Lynch, 1987; Garcia-Molina, 1982; and Singh and Kurose, 1994).

3.3.1. The Bully Algorithm

As a first example, consider the **bully algorithm** devised by Garcia-Molina (1982). When a process notices that the coordinator is no longer responding to requests, it initiates an election. A process, P , holds an election as follows:

1. P sends an *ELECTION* message to all processes with higher numbers.
2. If no one responds, P wins the election and becomes coordinator.
3. If one of the higher-ups answers, it takes over. P 's job is done.

At any moment, a process can get an *ELECTION* message from one of its lower-numbered colleagues. When such a message arrives, the receiver sends an *OK* message back to the sender to indicate that he is alive and will take over. The receiver then holds an election, unless it is already holding one. Eventually, all processes give up but one, and that one is the new coordinator. It announces its victory by sending all processes a message telling them that starting immediately it is the new coordinator.

If a process that was previously down comes back up, it holds an election. If it happens to be the highest-numbered process currently running, it will win the election and take over the coordinator's job. Thus the biggest guy in town always wins, hence the name "bully algorithm."

In Fig. 3-12 we see an example of how the bully algorithm works. The group consists of eight processes, numbered from 0 to 7. Previously process 7 was the coordinator, but it has just crashed. Process 4 is the first one to notice this, so it sends *ELECTION* messages to all the processes higher than it, namely 5, 6, and 7, as shown in Fig. 3-12(a). Processes 5 and 6 both respond with *OK*, as shown in Fig. 3-12(b). Upon getting the first of these responses, 4 knows that its job is over. It knows that one of these bigwigs will take over and become coordinator. It just sits back and waits to see who the winner will be (although at this point it can make a pretty good guess).

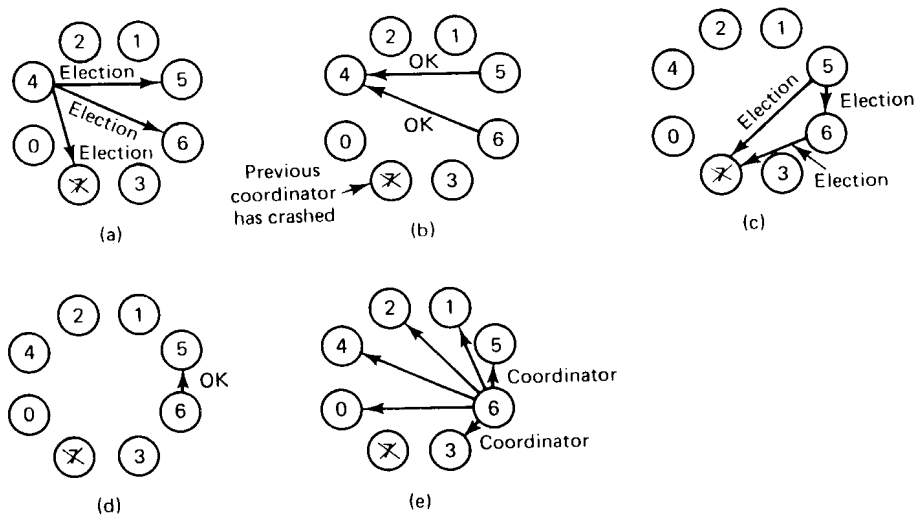


Fig. 3-12. The bully election algorithm. (a) Process 4 holds an election. (b) Processes 5 and 6 respond, telling 4 to stop. (c) Now 5 and 6 each hold an election. (d) Process 6 tells 5 to stop. (e) Process 6 wins and tells everyone.

In Fig. 3-13(c), both 5 and 6 hold elections, each one only sending messages to those processes higher than itself. In Fig. 3-13(d) process 6 tells 5 that it will take over. At this point 6 knows that 7 is dead and that it (6) is the winner. If there is state information to be collected from disk or elsewhere to pick up where the old coordinator left off, 6 must now do what is needed. When it is ready to take over, 6 announces this by sending a *COORDINATOR* message to all running processes. When 4 gets this message, it can now continue with the operation it was trying to do when it discovered that 7 was dead, but using 6 as the coordinator this time. In this way the failure of 7 is handled and the work can continue.

If process 7 is ever restarted, it will just send all the others a *COORDINATOR* message and bully them into submission.

3.3.2. A Ring Algorithm

Another election algorithm is based on the use of a ring, but without a token. We assume that the processes are physically or logically ordered, so that each process knows who its successor is. When any process notices that the coordinator is not functioning, it builds an *ELECTION* message containing its own process number and sends the message to its successor. If the successor is down, the sender skips over the successor and goes to the next member along the ring, or the one after that, until a running process is located. At each step, the sender adds its own process number to the list in the message.

Eventually, the message gets back to the process that started it all. That process recognizes this event when it receives an incoming message containing its own process number. At that point, the message type is changed to *COORDINATOR* and circulated once again, this time to inform everyone else who the coordinator is (the list member with the highest number) and who the members of the new ring are. When this message has circulated once, it is removed and everyone goes back to work.

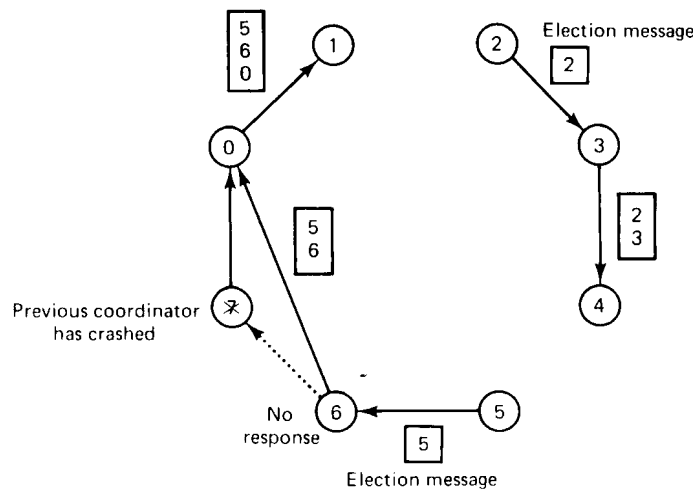


Fig. 3-13. Election algorithm using a ring.

In Fig. 3-13 we see what happens if two processes, 2 and 5, discover simultaneously that the previous coordinator, process 7, has crashed. Each of these builds an *ELECTION* message and starts circulating it. Eventually, both messages will go all the way around, and both 2 and 5 will convert them into *COORDINATOR* messages, with exactly the same members and in the same order. When both have gone around again, both will be removed. It does no harm to have extra messages circulating; at most it wastes a little bandwidth.

3.4. ATOMIC TRANSACTIONS

All the synchronization techniques we have studied so far are essentially low level, like semaphores. They require the programmer to be intimately involved with all the details of mutual exclusion, critical region management, deadlock prevention, and crash recovery. What we would really like is a much higher-level abstraction, one that hides these technical issues and allows the programmer to concentrate on the algorithms and how the processes work together in parallel. Such an abstraction exists and is widely used in distributed systems. We will call it an **atomic transaction**, or simply **transaction**. The term **atomic action** is also widely used. In this section we will examine the use, design, and implementation of atomic transactions.

3.4.1. Introduction to Atomic Transactions

The original model of the atomic transaction comes from the world of business. Suppose that the International Dingbat Corporation needs a batch of widgets. They approach a potential supplier, U.S. Widget, known far and wide for the quality of its widgets, for a quote on 100,000 10-cm purple widgets for June delivery. U.S. Widget makes a bid on 100,000 4-inch mauve widgets to be delivered in December. International Dingbat agrees to the price, but dislikes mauve, wants them by July, and insists on 10 cm for its international customers. U.S. Widget replies by offering 3 15/16 inch lavender widgets in October. After much further negotiation, they finally agree on 3 959/1024 inch violet widgets for delivery on August 15.

Up until this point, both parties are free to terminate the discussion, in which case the world returns to the state it was in before they started talking. However, once both companies have signed a contract, they are both legally bound to complete the sale, come what may. Thus until both parties have signed on the dotted line, either one can back out and it is as if nothing ever happened, but at the moment they both sign, they pass the point of no return and the transaction must be carried out.

The computer model is similar. One process announces that it wants to begin a transaction with one or more other processes. They can negotiate various options, create and delete objects, and perform operations for a while. Then the initiator announces that it wants all the others to commit themselves to the work done so far. If all of them agree, the results are made permanent. If one or more processes refuse (or crash before agreement), the situation reverts to exactly the state it had before the transaction began, with all side effects on objects, files, data bases, and so on, magically wiped out. This all-or-nothing property eases the programmer's job.

The use of transactions in computer systems goes back to the 1960s. Before

there were disks and online data bases, all files were kept on magnetic tape. Imagine a supermarket with an automated inventory system. Every day after closing, a computer run was made with two input tapes. The first one contained the complete inventory as of opening time that morning. The second one contained a list of the day's updates: products sold to customers and products delivered by suppliers. The computer read both input tapes and produced a new master inventory tape, as shown in Fig. 3-14.

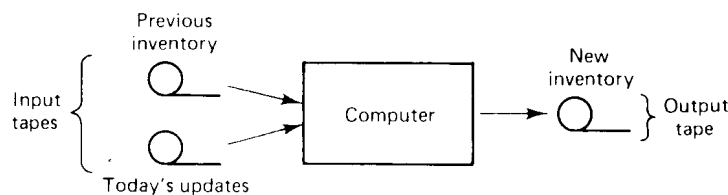


Fig. 3-14. Updating a master tape is fault tolerant.

The great beauty of this scheme (although the people who had to live with it did not realize that) is that if a run failed for any reason, all the tapes could be rewound and the job restarted with no harm done. Primitive as it was, the old magnetic tape system had the all-or-nothing property of an atomic transaction.

Now look at a modern banking application that updates an online data base in place. The customer calls up the bank using a PC with a modem with the intention of withdrawing money from one account and depositing it in another. The operation is performed in two steps:

1. Withdraw(amount, account1).
2. Deposit(amount, account2).

If the telephone connection is broken after the first one but before the second one, the first account will have been debited but the second one will not have been credited. The money vanishes into thin air.

Being able to group these two operations in an atomic transaction would solve the problem. Either both would be completed, or neither would be completed. The key is rolling back to the initial state if the transaction fails to complete. What we really want is a way to rewind the data base as we could the magnetic tapes. This ability is what the atomic transaction has to offer.

3.4.2. The Transaction Model

We will now develop a more precise model of what a transaction is and what its properties are. The system is assumed to consist of some number of independent processes, each of which can fail at random. Communication is

normally unreliable in that messages can be lost, but lower levels can use a timeout and retransmission protocol to recover from lost messages. Thus for this discussion we can assume that communication errors are handled transparently by underlying software.

Stable Storage

Storage comes in three categories. First we have ordinary RAM memory, which is wiped out when the power fails or a machine crashes. Next we have disk storage, which survives CPU failures but which can be lost in disk head crashes.

Finally, we have **stable storage**, which is designed to survive anything except major calamities such as floods and earthquakes. Stable storage can be implemented with a pair of ordinary disks, as shown in Fig. 3-15(a). Each block on drive 2 is an exact copy of the corresponding block on drive 1. When a block is updated, first the block on drive 1 is updated and verified, then the same block on drive 2 is done.

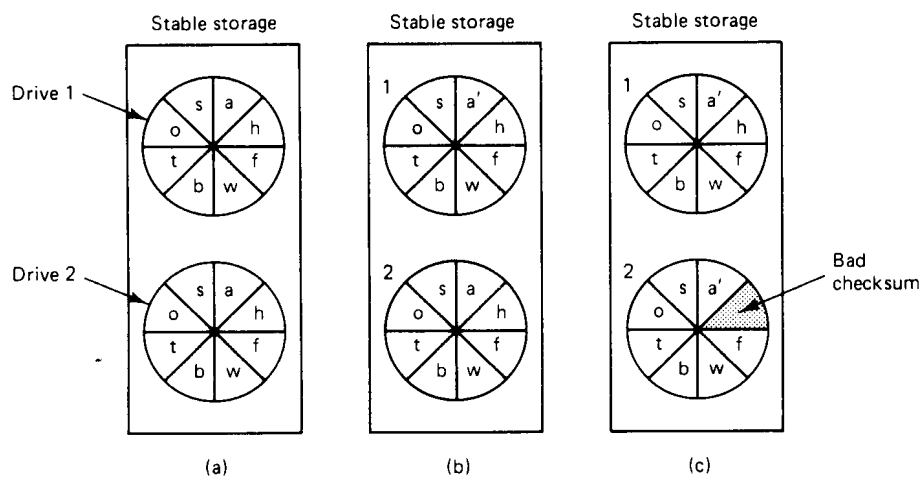


Fig. 3-15. (a) Stable storage. (b) Crash after drive 1 is updated. (c) Bad spot.

Suppose that the system crashes after drive 1 is updated but before drive 2 is updated, as shown in Fig. 3-15(b). Upon recovery, the disk can be compared block for block. Whenever two corresponding blocks differ, it can be assumed that drive 1 is the correct one (because drive 1 is always updated before drive 2), so the new block is copied from drive 1 to drive 2. When the recovery process is complete, both drives will again be identical.

Another potential problem is the spontaneous decay of a block. Dust

particles or general wear and tear can give a previously valid block a sudden checksum error, without cause or warning, as shown in Fig. 3-15(c). When such an error is detected, the bad block can be regenerated from the corresponding block on the other drive.

As a consequence of its implementation, stable storage is well suited to applications that require a high degree of fault tolerance, such as atomic transactions. When data are written to stable storage and then read back to check that they have been written correctly, the chance of them subsequently being lost is extremely small.

Transaction Primitives

Programming using transactions requires special primitives that must either be supplied by the operating system or by the language runtime system. Examples are:

1. `BEGIN_TRANSACTION`: Mark the start of a transaction.
2. `END_TRANSACTION`: Terminate the transaction and try to commit.
3. `ABORT_TRANSACTION`: Kill the transaction; restore the old values.
4. `READ`: Read data from a file (or other object).
5. `WRITE`: Write data to a file (or other object).

The exact list of primitives depends on what kinds of objects are being used in the transaction. In a mail system, there might be primitives to send, receive, and forward mail. In an accounting system, they might be quite different. `READ` and `WRITE` are typical examples, however. Ordinary statements, procedure calls, and so on, are also allowed inside a transaction.

`BEGIN_TRANSACTION` and `END_TRANSACTION` are used to delimit the scope of a transaction. The operations between them form the body of the transaction. Either all of them are executed or none are executed. These may be system calls, library procedures, or bracketing statements in a language, depending on the implementation.

Consider, for example, the process of reserving a seat from White Plains, New York, to Malindi, Kenya, in an airline reservation system. One route is White Plains to JFK, JFK to Nairobi, and Nairobi to Malindi. In Fig. 3-16(a) we see reservations for these three separate flights being made as three actions. Now suppose that the first two flights have been reserved but the third one is booked solid. The transaction is aborted and the results of the first two bookings are undone—the airline data base is restored to the value it had before the transaction started [see Fig. 3-16(b)]. It is as though nothing happened.

<pre>BEGIN_TRANSACTION reserve WP-JFK; reserve JFK-Nairobi; reserve Nairobi-Malindi; END_TRANSACTION</pre>	<pre>BEGIN_TRANSACTION reserve WP-JFK; reserve JFK-Nairobi; Nairobi-Malindi full ⇒ ABORT_TRANSACTION;</pre>
(a)	(b)

Fig. 3-16. (a) Transaction to reserve three flights commits. (b) Transaction aborts when third flight is unavailable.

Properties of Transactions

Transactions have four essential properties. Transactions are:

1. Atomic: To the outside world, the transaction happens indivisibly.
2. Consistent: The transaction does not violate system invariants.
3. Isolated: Concurrent transactions do not interfere with each other.
4. Durable: Once a transaction commits, the changes are permanent.

These properties are often referred to by their initial letters, **ACID**.

The first key property exhibited by all transactions is that they are **atomic**. This property ensures that each transaction either happens completely, or not at all, and if it happens, it happens in a single indivisible, instantaneous action. While a transaction is in progress, other processes (whether or not they are themselves involved in transactions) cannot see any of the intermediate states.

Suppose, for example, that some file is 10 bytes long when a transaction starts to append to it. If other processes read the file while the transaction is in progress, they see only the original 10 bytes, no matter how many bytes the transaction has appended. If the transaction commits successfully, the file grows instantaneously to its new size at the moment of commitment, with no intermediate states, no matter how many operations it took to get it there.

The second property says that they are **consistent**. What this means is that if the system has certain invariants that must always hold, if they held before the transaction, they will hold afterward too. For example, in a banking system, a key invariant is the law of conservation of money. After any internal transfer, the amount of money in the bank must be the same as it was before the transfer, but for a brief moment during the transaction, this invariant may be violated. The violation is not visible outside the transaction, however.

The third property says that transactions are **isolated** or **serializable**. What it means is that if two or more transactions are running at the same time, to each of them and to other processes, the final result looks as though all transactions ran sequentially in some (system dependent) order.

In Fig. 3-17(a)-(c) we have three transactions that are executed simultaneously by three separate processes. If they were to be run sequentially, the final value of x would be 1, 2, or 3, depending which one ran last (x could be a shared variable, a file, or some other kind of object). In Fig. 3-17(d) we see various orders, called **schedules**, in which they might be interleaved. Schedule 1 is actually serialized. In other words, the transactions run strictly sequentially, so it meets the serializability condition by definition. Schedule 2 is not serialized, but is still legal because it results in a value for x that could have been achieved by running the transactions strictly sequentially. The third one is illegal since it sets x to 5, something that no sequential order of the transactions could produce. It is up to the system to ensure that individual operations are interleaved correctly. By allowing the system the freedom to choose any ordering of the operations it wants to—provided that it gets the answer right—we eliminate the need for programmers to do their own mutual exclusion, thus simplifying the programming.

BEGIN_TRANSACTION

$x = 0;$

$x = x + 1;$

END_TRANSACTION

(a)

BEGIN_TRANSACTION

$x = 0;$

$x = x + 2;$

END_TRANSACTION

(b)

BEGIN_TRANSACTION

$x = 0;$

$x = x + 3;$

END_TRANSACTION

(c)

Time $\longrightarrow$

Schedule 1	$x = 0;$	$x = x + 1;$	$x = 0;$	$x = x + 2;$	$x = 0;$	$x = x + 3;$	Legal
Schedule 2	$x = 0;$	$x = 0;$	$x = x + 1;$	$x = x + 2;$	$x = 0;$	$x = x + 3;$	Legal
Schedule 3	$x = 0;$	$x = 0;$	$x = x + 1;$	$x = 0;$	$x = x + 2;$	$x = x + 3;$	Illegal

(d)

(d)

Fig. 3-17. (a)–(c) Three transactions. (d) Possible schedules.

The fourth property says that transactions are **durable**. It refers to the fact that once a transaction commits, no matter what happens, the transaction goes forward and the results become permanent. No failure after the commit can undo the results or cause them to be lost.

Nested Transactions

Transactions may contain subtransactions, often called **nested transactions**. The top-level transaction may fork off children that run in parallel with one another, on different processors, to gain performance or simplify programming.

Each of these children may execute one or more subtransactions, or fork off its own children.

Subtransactions give rise to a subtle, but important, problem. Imagine that a transaction starts several subtransactions in parallel, and one of these commits, making its results visible to the parent transaction. After further computation, the parent aborts, restoring the entire system to the state it had before the top-level transaction started. Consequently, the results of the subtransaction that committed must nevertheless be undone. Thus the permanence referred to above applies only to top-level transactions.

Since transactions can be nested arbitrarily deeply, considerable administration is needed to get everything right. The semantics are clear, however. When any transaction or subtransaction starts, it is conceptually given a private copy of all objects in the entire system for it to manipulate as it wishes. If it aborts, its private universe just vanishes, as if it had never existed. If it commits, its private universe replaces the parent's universe. Thus if a subtransaction commits and then later a new subtransaction is started, the second one sees the results produced by the first one.

3.4.3. Implementation

Transactions sound like a great idea, but how are they implemented? That is the question we will tackle in this section. It should be clear by now that if each process executing a transaction just updates the objects it uses (files, data base records, etc.) in place, transactions will not be atomic and changes will not vanish magically if the transaction aborts. Furthermore, the results of running multiple transactions will not be serializable either. Clearly, some other implementation method is required. Two methods are commonly used. They will be discussed in turn below.

Private Workspace

Conceptually, when a process starts a transaction, it is given a private workspace containing all the files (and other objects) to which it has access. Until the transaction either commits or aborts, all of its reads and writes go to the private workspace, rather than the "real" one, by which we mean the normal file system. This observation leads directly to the first implementation method: actually giving a process a private workspace at the instant it begins a transaction.

The problem with this technique is that the cost of copying everything to a private workspace is prohibitive, but various optimizations make it feasible. The first optimization is based on the realization that when a process reads a file but does not modify it, there is no need for a private copy. It can just use the

real one (unless it has been changed since the transaction started). Consequently, when a process starts a transaction, it is sufficient to create a private workspace for it that is empty except for a pointer back to its parent's workspace. When the transaction is at the top level, the parent's workspace is the "real" file system. When the process opens a file for reading, the back pointers are followed until the file is located in the parent's (or further ancestor's) workspace.

When a file is opened for writing, it can be located in the same way as for reading, except that now it is first copied to the private workspace. However, a second optimization removes most of the copying, even here. Instead of copying the entire file, only the file's index is copied into the private workspace. The index is the block of data associated with each file telling where its disk blocks are. In UNIX, the index is the i-node. Using the private index, the file can be read in the usual way, since the disk addresses it contains are for the original disk blocks. However, when a file block is first modified, a copy of the block is made and the address of the copy inserted into the index, as shown in Fig. 3-18. The block can then be updated without affecting the original. Appended blocks are handled this way too. The new blocks are sometimes called **shadow blocks**.

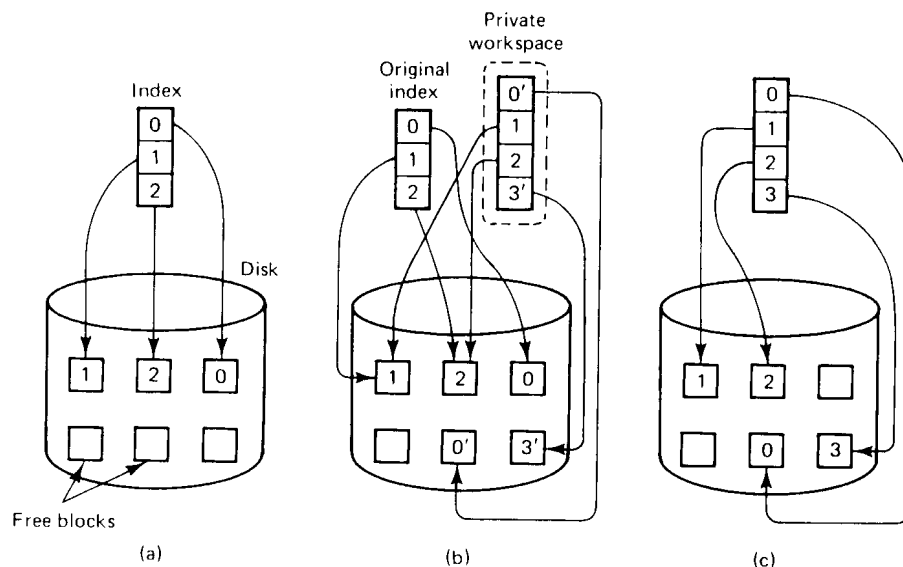


Fig. 3-18. (a) The file index and disk blocks for a three-block file. (b) The situation after a transaction has modified block 0 and appended block 3. (c) After committing.

As can be seen from Fig. 3-18(b), the process running the transaction sees the modified file, but all other processes continue to see the original file. In a

more complex transaction, the private workspace might contain a large number of files instead of just one. If the transaction aborts, the private workspace is simply deleted and all the private blocks that it points to are put back on the free list. If the transaction commits, the private indices are moved into the parent's workspace atomically, as shown in Fig. 3-18(c). The blocks that are no longer reachable are put onto the free list.

Writeahead Log

The other common method of implementing transactions is the **writeahead log**, sometimes called an **intentions list**. With this method, files are actually modified in place, but before any block is changed, a record is written to the writeahead log on stable storage telling which transaction is making the change, which file and block is being changed, and what the old and new values are. Only after the log has been written successfully is the change made to the file.

Figure 3-19 gives an example of how the log works. In Fig. 3-19(a) we have a simple transaction that uses two shared variables (or other objects), x and y , both initialized to 0. For each of the three statements inside the transaction, a log record is written before executing the statement, giving the old and new values, separated by a slash.

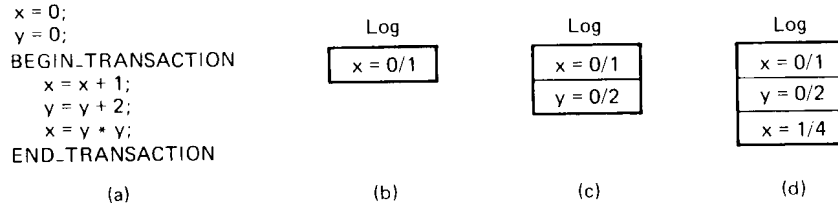


Fig. 3-19. (a) A transaction. (b)–(d) The log before each statement is executed.

If the transaction succeeds and is committed, a commit record is written to the log, but the data structures do not have to be changed, as they have already been updated. If the transaction aborts, the log can be used to back up to the original state. Starting at the end and going backward, each log record is read and the change described in it undone. This action is called a **rollback**.

The log can also be used for recovering from crashes. Suppose that the process doing the transaction crashes just after having written the last log record of Fig. 3-19(d), but before changing x . After the failed machine is rebooted, the log is checked to see if any transactions were in progress at the time of the crash. When the last record is read and the current value of x is seen to be 1, it is clear that the crash occurred *before* the update was made, so x is set to 4. If, on the other hand, x is 4 at the time of recovery, it is equally clear that the crash

occurred *after* the update, so nothing need be changed. Using the log, it is possible to go forward (do the transaction) or go backward (undo the transaction).

Two-Phase Commit Protocol

As we have pointed out repeatedly, the action of committing a transaction must be done atomically, that is, instantaneously and indivisibly. In a distributed system, the commit may require the cooperation of multiple processes on different machines, each of which holds some of the variables, files, and data bases, and other objects changed by the transaction. In this section we will study a protocol for achieving atomic commit in a distributed system.

The protocol we will look at is called the **two-phase commit protocol** (Gray, 1978). Although it is not the only such protocol, it is probably the most widely used. The basic idea is illustrated in Fig. 3-20. One of the processes involved functions as the coordinator. Usually, this is the one executing the transaction. The commit protocol begins when the coordinator writes a log entry saying that it is starting the commit protocol, followed by sending each of the other processes involved (the subordinates) a message telling them to prepare to commit.

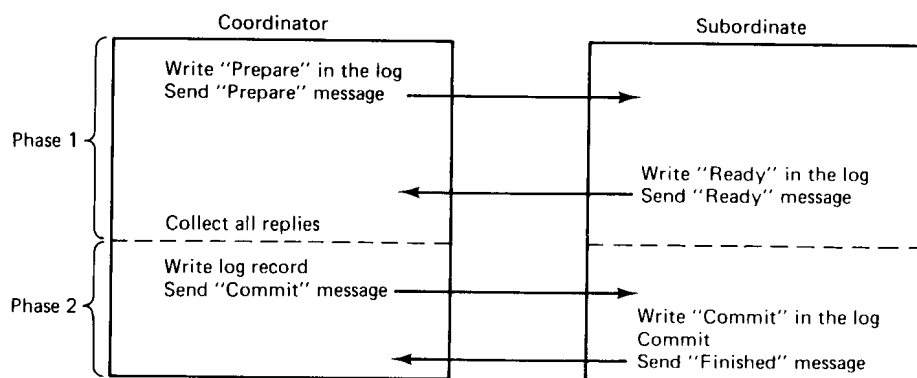


Fig. 3-20. The two-phase commit protocol when it succeeds.

When a subordinate gets the message it checks to see if it is ready to commit, makes a log entry, and sends back its decision. When the coordinator has received all the responses, it knows whether to commit or abort. If all the processes are prepared to commit, the transaction is committed. If one or more are unable to commit (or do not respond), the transaction is aborted. Either way, the coordinator writes a log entry and then sends a message to each subordinate informing it of the decision. It is this write to the log that actually commits the transaction and makes it go forward no matter what happens afterward.

Due to the use of the log on stable storage, this protocol is highly resilient in the face of (multiple) crashes. If the coordinator crashes after having written the initial log record, upon recovery it can just continue where it left off, repeating the initial message if need be. If it crashes after having written the result of the vote to the log, upon recovery it can just reinform all the subordinates of the result. If a subordinate crashes before having replied to the first message, the coordinator will keep sending it messages, until it gives up. If it crashes later, it can see from the log where it was, and thus what it must do.

3.4.4. Concurrency Control

When multiple transactions are executing simultaneously in different processes (on different processors), some mechanism is needed to keep them out of each other's way. That mechanism is called a **concurrency control algorithm**. In this section we will study three different ones.

Locking

The oldest and most widely used concurrency control algorithm is **locking**. In the simplest form, when a process needs to read or write a file (or other object) as part of a transaction, it first locks the file. Locking can be done using a single centralized lock manager, or with a local lock manager on each machine for managing local files. In both cases the lock manager maintains a list of locked files, and rejects all attempts to lock files that are already locked by another process. Since well-behaved processes do not attempt to access a file before it has been locked, setting a lock on a file keeps everyone else away from it and thus ensures that it will not change during the lifetime of the transaction. Locks are normally acquired and released by the transaction system and do not require action by the programmer.

This basic scheme is overly restrictive and can be improved by distinguishing read locks from write locks. If a read lock is set on a file, other read locks are permitted. Read locks are set to make sure that the file does not change (i.e., exclude all writers), but there is no reason to forbid other transactions from reading the file. In contrast, when a file is locked for writing, no other locks of any kind are permitted. Thus read locks are shared, but write locks must be exclusive.

For simplicity, we have assumed that the unit of locking is the entire file. In practice, it might be a smaller item, such as an individual record or page, or a larger item, such as an entire data base. The issue of how large an item to lock is called the **granularity of locking**. The finer the granularity, the more precise the lock can be, and the more parallelism can be achieved (e.g., by not blocking

a process that wants to use the end of a file just because some other process is using the beginning). On the other hand, fine-grained locking requires more locks, is more expensive, and is more likely to lead to deadlocks.

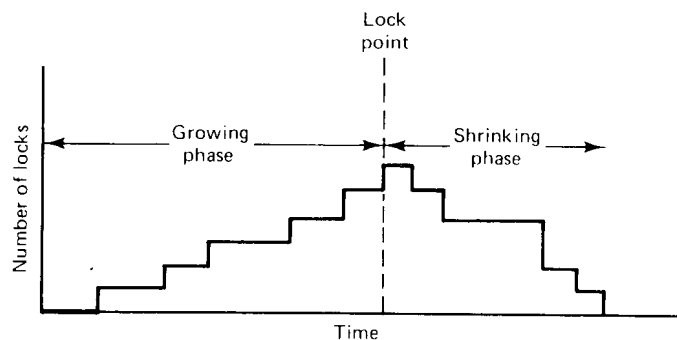


Fig. 3-21. Two-phase locking.

Acquiring and releasing locks precisely at the moment they are needed or no longer needed can lead to inconsistency and deadlocks. Instead, most transactions that are implemented by locking use what is called **two-phase locking**. In two-phase locking, which is illustrated in Fig. 3-21, the process first acquires all the locks it needs during the **growing phase**, then releases them during the **shrinking phase**. If the process refrains from updating any files until it reaches the shrinking phase, failure to acquire some lock can be dealt with simply by releasing all locks, waiting a little while, and starting all over. Furthermore, it can be proven (Eswaran et al., 1976) that if all transactions use two-phase locking, all schedules formed by interleaving them are serializable. This is why two-phase locking is widely used.

In many systems, the shrinking phase does not take place until the transaction has finished running and has either committed or aborted. This policy, called **strict two-phase locking**, has two main advantages. First, a transaction always reads a value written by a committed transaction; therefore, one never has to abort a transaction because its calculations were based on a file it should not have seen. Second, all lock acquisitions and releases can be handled by the system without the transaction being aware of them: locks are acquired whenever a file is to be accessed and released when the transaction has finished. This policy eliminates **cascaded aborts**: having to undo a committed transaction because it saw a file it should not have seen.

Locking, even two-phase locking, can lead to deadlocks. If two processes each try to acquire the same pair of locks but in the opposite order, a deadlock may result. The usual techniques apply here, such as acquiring all locks in some canonical order to prevent hold-and-wait cycles. Also possible is deadlock

detection by maintaining an explicit graph of which process has which locks and wants which locks, and checking the graph for cycles. Finally, when it is known in advance that a lock will never be held longer than T sec, a timeout scheme can be used: if a lock remains continuously under the same ownership for longer than T sec, there must be a deadlock.

Optimistic Concurrency Control

A second approach to handling multiple transactions at the same time is **optimistic concurrency control** (Kung and Robinson, 1981). The idea behind this technique is surprisingly simple: just go ahead and do whatever you want to, without paying attention to what anybody else is doing. If there is a problem, worry about it later. (Many politicians use this algorithm, too.) In practice, conflicts are relatively rare, so most of the time it works all right.

Although conflicts may be rare, they are not impossible, so some way is needed to handle them. What optimistic concurrency control does is keep track of which files have been read and written. At the point of committing, it checks all other transactions to see if any of its files have been changed since the transaction started. If so, the transaction is aborted. If not, it is committed.

Optimistic concurrency control fits best with the implementation based on private workspaces. That way, each transaction changes its files privately, without interference from the others. At the end, the new files are either committed or released.

The big advantages of optimistic concurrency control are that it is deadlock free and allows maximum parallelism because no process ever has to wait for a lock. The disadvantage is that sometimes it may fail, in which case the transaction has to be run all over again. Under conditions of heavy load, the probability of failure may go up substantially, making optimistic concurrency control a poor choice.

Timestamps

A completely different approach to concurrency control is to assign each transaction a timestamp at the moment it does `BEGIN_TRANSACTION` (Reed, 1983). Using Lamport's algorithm, we can ensure that the timestamps are unique, which is important here. Every file in the system has a read timestamp and a write timestamp associated with it, telling which committed transaction last read and wrote it, respectively. If transactions are short and widely spaced in time, it will normally occur that when a process tries to access a file, the file's read and write timestamps will be lower (older) than the current transaction's

timestamp. This ordering means that the transactions are being processed in the proper order, so everything is all right.

When the ordering is incorrect, it means that a transaction that started later than the current one has managed to get in there, access the file, and commit. This situation means that the current transaction is too late, so it is aborted. In a sense, this mechanism is also optimistic, like that of Kung and Robinson, although the details are quite different. In Kung and Robinson's method, we are hoping that concurrent transactions do not use the same files. In the timestamp method, we do not mind if concurrent transactions use the same files, as long as the lower numbered transaction always goes first.

It is easiest to explain the timestamp method by means of an example. Imagine that there are three transactions, alpha, beta, and gamma. Alpha ran a long time ago, and used every file needed by beta and gamma, so all their files have read and write timestamps set to alpha's timestamp. Beta and gamma start concurrently, with beta having a lower timestamp than gamma (but higher than alpha, of course).

Let us first consider beta writing a file. Call its timestamp, T , and the read and write timestamps of the file to be written T_{RD} and T_{WR} , respectively. Unless gamma has snuck in already and committed, both T_{RD} and T_{WR} will be alpha's timestamp, and thus less than T . In Fig. 3-22(a) and (b) we see that T is larger than both T_{RD} and T_{WR} (gamma has not already committed), so the write is accepted and done tentatively. It will become permanent when beta commits. Beta's timestamp is now recorded in the file as a tentative write.

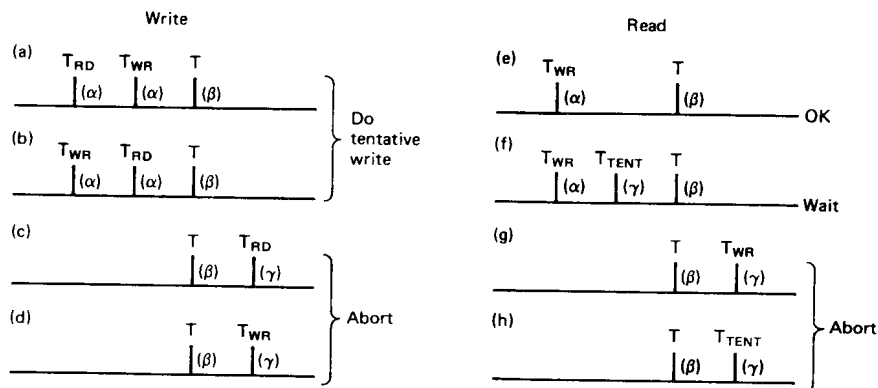


Fig. 3-22. Concurrency control using timestamps.

In Fig. 3-22(c) and (d) beta is out of luck. Gamma has either read (c) or written (d) the file and committed. Beta's transaction is aborted. However, it can apply for a new timestamp and start all over again.

Now look at reads. In Fig. 3-22(e), there is no conflict, so the read can

happen immediately. In Fig. 3-22(f), some interloper has gotten in there and is trying to write the file. The interloper's timestamp is lower than beta's, so beta simply waits until the interloper commits, at which time it can read the new file and continue.

In Fig. 3-22(g), gamma has changed the file and already committed. Again beta must abort. In Fig. 3-22(h), gamma is in the process of changing the file, although it has not committed yet. Still, beta is too late and must abort.

Timestamping has different properties than locking. When a transaction encounters a larger (later) timestamp, it aborts, whereas under the same circumstances with locking it would either wait or be able to proceed immediately. On the other hand, it is deadlock free, which is a big plus.

All in all, transactions offer many advantages and thus are a promising technique for building reliable distributed systems. Their chief problem is their great implementation complexity, which yields low performance. These problems are being worked on, and perhaps in due course they will be solved.

3.5. DEADLOCKS IN DISTRIBUTED SYSTEMS

Deadlocks in distributed systems are similar to deadlocks in single-processor systems, only worse. They are harder to avoid, prevent, or even detect, and harder to cure when tracked down because all the relevant information is scattered over many machines. In some systems, such as distributed data base systems, they can be extremely serious, so it is important to understand how they differ from ordinary deadlocks and what can be done about them.

Some people make a distinction between two kinds of distributed deadlocks: communication deadlocks and resource deadlocks. A communication deadlock occurs, for example, when process *A* is trying to send a message to process *B*, which in turn is trying to send one to process *C*, which is trying to send one to *A*. There are various scenarios in which this situation leads to deadlock, such as no buffers being available. A resource deadlock occurs when processes are fighting over exclusive access to I/O devices, files, locks, or other resources.

We will not make that distinction here, since communication channels, buffers, and so on, are also resources and can be modeled as resource deadlocks because processes can request them and release them. Furthermore, circular communication patterns of the type just described are quite rare in most systems. In client-server systems, for example, a client might send a message (or perform an RPC) with a file server, which might send a message to a disk server. However, it is unlikely that the disk server, acting as a client, would send a message to the original client, expecting it to act like a server. Thus the circular wait condition is unlikely to occur as a result of communication alone.

Various strategies are used to handle deadlocks. Four of the best-known ones are listed and discussed below.

1. The ostrich algorithm (ignore the problem).
2. Detection (let deadlocks occur, detect them, and try to recover).
3. Prevention (statically make deadlocks structurally impossible).
4. Avoidance (avoid deadlocks by allocating resources carefully).

All four are potentially applicable to distributed systems. The ostrich algorithm is as good and as popular in distributed systems as it is in single-processor systems. In distributed systems used for programming, office automation, process control, and many other applications, no system-wide deadlock mechanism is present, although individual applications, such as distributed data bases, can implement their own if they need one.

Deadlock detection and recovery is also popular, primarily because prevention and avoidance are so difficult. We will discuss several algorithms for deadlock detection below.

Deadlock prevention is also possible, although more difficult than in single-processor systems. However, in the presence of atomic transactions, some new options become available. Two algorithms are discussed below.

Finally, deadlock avoidance is never used in distributed systems. It is not even used in single-processor systems, so why should it be used in the more difficult case of distributed systems? The problem is that the banker's algorithm and similar algorithms need to know (in advance) how much of each resource every process will eventually need. This information is rarely, if ever, available. Thus our discussion of deadlocks in distributed systems will focus on just two of the techniques: deadlock detection and deadlock prevention.

3.5.1. Distributed Deadlock Detection

Finding general methods for preventing or avoiding distributed deadlocks appears to be quite difficult, so many researchers have tried to deal with the simpler problem of just detecting deadlocks, rather than trying to inhibit their occurrence.

However, the presence of atomic transactions in some distributed systems makes a major conceptual difference. When a deadlock is detected in a conventional operating system, the way to resolve it is to kill off one or more processes. Doing so invariably leads to one or more unhappy users. When a deadlock is detected in a system based on atomic transactions, it is resolved by aborting one or more transactions. But as we have seen in detail above, transactions have been designed to withstand being aborted. When a transaction is aborted

because it contributes to a deadlock, the system is first restored to the state it had before the transaction began, at which point the transaction can start again. With a little bit of luck, it will succeed the second time. Thus the difference is that the consequences of killing off a process are much less severe when transactions are used than when they are not used.

Centralized Deadlock Detection

As a first attempt, we can use a centralized deadlock detection algorithm and try to imitate the nondistributed algorithm. Although each machine maintains the resource graph for its own processes and resources, a central coordinator maintains the resource graph for the entire system (the union of all the individual graphs). When the coordinator detects a cycle, it kills off one process to break the deadlock.

Unlike the centralized case, where all the information is automatically available in the right place, in a distributed system it has to be sent there explicitly. Each machine maintains the graph for its own processes and resources. Several possibilities exist for getting it there. First, whenever an arc is added or deleted from the resource graph, a message can be sent to the coordinator providing the update. Second, periodically, every process can send a list of arcs added or deleted since the previous update. This method requires fewer messages than the first one. Third, the coordinator can ask for information when it needs it.

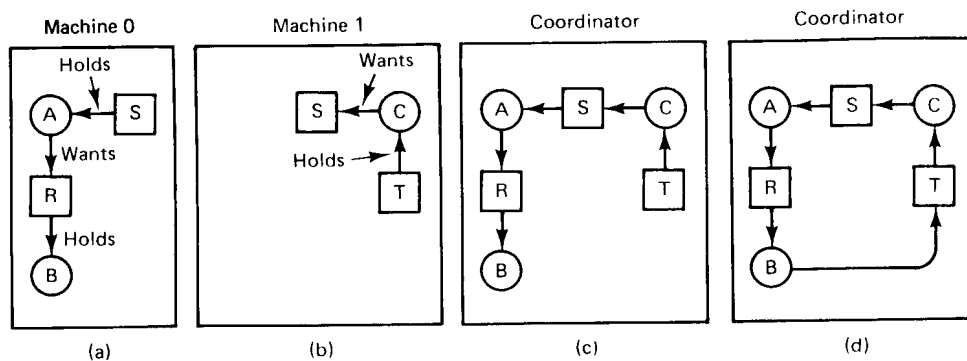


Fig. 3-23. (a) Initial resource graph for machine 0. (b) Initial resource graph for machine 1. (c) The coordinator's view of the world. (d) The situation after the delayed message.

Unfortunately, none of these methods work well. Consider a system with processes *A* and *B* running on machine 0, and process *C* running on machine 1. Three resources exist: *R*, *S*, and *T*. Initially, the situation is as shown in Fig. 3-23(a) and (b): *A* holds *S* but wants *R*, which it cannot have because *B* is using it;

C has *T* and wants *S*, too. The coordinator's view of the world is shown in Fig. 3-23(c). This configuration is safe. As soon as *B* finishes, *A* can get *R* and finish, releasing *S* for *C*.

After a while, *B* releases *R* and asks for *T*, a perfectly legal and safe swap. Machine 0 sends a message to the coordinator announcing the release of *R*, and machine 1 sends a message to the coordinator announcing the fact that *B* is now waiting for its resource, *T*. Unfortunately, the message from machine 1 arrives first, leading the coordinator to construct the graph of Fig. 3-23(d). The coordinator incorrectly concludes that a deadlock exists and kills some process. Such a situation is called a **false deadlock**. Many deadlock algorithms in distributed systems produce false deadlocks like this due to incomplete or delayed information.

One possible way out might be to use Lamport's algorithm to provide global time. Since the message from machine 1 to the coordinator is triggered by the request from machine 0, the message from machine 1 to the coordinator will indeed have a later timestamp than the message from machine 0 to the coordinator. When the coordinator gets the message from machine 1 that leads it to suspect deadlock, it could send a message to every machine in the system saying: "I just received a message with timestamp *T* which leads to deadlock. If anyone has a message for me with an earlier timestamp, please send it immediately." When every machine has replied, positively or negatively, the coordinator will see that the arc from *R* to *B* has vanished, so the system is still safe. Although this method eliminates the false deadlock, it requires global time and is expensive. Furthermore, other situations exist where eliminating false deadlock is much harder.

Distributed Deadlock Detection

Many distributed deadlock detection algorithms have been published. Surveys of the subject are given in Knapp (1987) and Singhal (1989). Let us examine a typical one here, the Chandy-Misra-Haas algorithm (Chandy et al., 1983). In this algorithm, processes are allowed to request multiple resources (e.g., locks) at once, instead of one at a time. By allowing multiple requests simultaneously, the growing phase of a transaction can be speeded up considerably. The consequence of this change to the model is that a process may now wait on two or more resources simultaneously.

In Fig. 3-24, we present a modified resource graph, where only the processes are shown. Each arc passes through a resource, as usual, but for simplicity the resources have been omitted from the figure. Notice that process 3 on machine 1 is waiting for two resources, one held by process 4 and one held by process 5.

Some of the processes are waiting for local resources, such as process 1, but

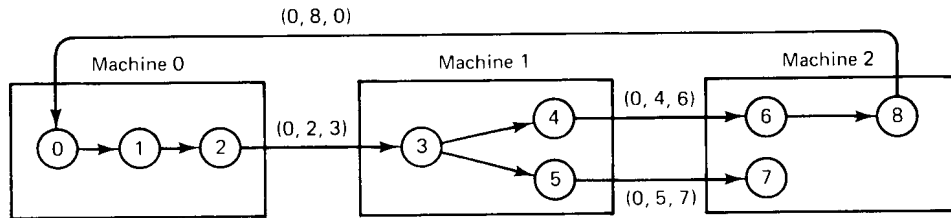


Fig. 3-24. The Chandy-Misra-Haas distributed deadlock detection algorithm.

others, such as process 2, are waiting for resources that are located on a different machine. It is precisely these cross-machine arcs that make looking for cycles difficult. The Chandy-Misra-Haas algorithm is invoked when a process has to wait for some resource, for example, process 0 blocking on process 1. At that point a special **probe** message is generated and sent to the process (or processes) holding the needed resources. The message consists of three numbers: the process that just blocked, the process sending the message, and the process to whom it is being sent. The initial message from 0 to 1 contains the triple (0, 0, 1).

When the message arrives, the recipient checks to see if it itself is waiting for any processes. If so, the message is updated, keeping the first field but replacing the second field by its own process number and the third one by the number of the process it is waiting for. The message is then sent to the process on which it is blocked. If it is blocked on multiple processes, all of them are sent (different) messages. This algorithm is followed whether the resource is local or remote. In Fig. 3-24 we see the remote messages labeled (0, 2, 3), (0, 4, 6), (0, 5, 7), and (0, 8, 0). If a message goes all the way around and comes back to the original sender, that is, the process listed in the first field, a cycle exists and the system is deadlocked.

There are various ways in which the deadlock can be broken. One way is to have the process that initiated the probe commit suicide. However, this method has problems if several processes invoke the algorithm simultaneously. In Fig. 3-24, for example, imagine that both 0 and 6 block at the same moment, and both initiate probes. Each would eventually discover the deadlock, and each would kill itself. This is overkill. Getting rid of one of them is enough.

An alternative algorithm is to have each process add its identity to the end of the probe message so that when it returned to the initial sender, the complete cycle would be listed. The sender can then see which process has the highest number, and kill that one or send it a message asking it to kill itself. Either way, if multiple processes discover the same cycle at the same time, they will all choose the same victim.

There are few areas of computer science in which theory and practice

diverge as much as in distributed deadlock detection algorithms. Discovering yet another deadlock detection algorithm is the goal of many a researcher. Unfortunately, these models often have little relation to reality. For example, some of the algorithms require processes to send probes when they are blocked. However, sending a probe when you are blocked is not entirely trivial.

Many of the papers contain elaborate analyses of the performance of the new algorithm, pointing out, for example, that while the new one requires two traversals of the cycle, it uses shorter messages, as if these factors balanced out somehow. The authors would no doubt be surprised to learn that a typical “short” message (20 bytes) on a LAN takes about 1 msec, and a typical “long” message (100 bytes) on the same LAN takes perhaps 1.1 msec. It would also no doubt come as a shock to these people to realize that experimental measurements have shown that 90 percent of all deadlock cycles involve exactly two processes (Gray et al., 1981).

Worst of all, a large fraction of all the published algorithms in this area are just plain wrong, including those proven to be correct. Knapp (1987) and Singhal (1989) point out some examples. It often occurs that shortly after an algorithm is invented, proven correct, and then published, somebody finds a counterexample. Thus we have an active research area in which the model of the problem does not correspond well to reality, the solutions found are generally impractical, the performance analyses given are meaningless, and the proven results are frequently incorrect. To end on a positive note, this is an area that offers great opportunities for improvement.

3.5.2. Distributed Deadlock Prevention

Deadlock prevention consists of carefully designing the system so that deadlocks are structurally impossible. Various techniques include allowing processes to hold only one resource at a time, requiring processes to request all their resources initially, and making processes release all resources when asking for a new one. All of these are cumbersome in practice. A method that sometimes works is to order all the resources and require processes to acquire them in strictly increasing order. This approach means that a process can never hold a high resource and ask for a low one, thus making cycles impossible.

However, in a distributed system with global time and atomic transactions, two other practical algorithms are possible. Both are based on the idea of assigning each transaction a global timestamp at the moment it starts. As in many timestamp-based algorithms, in these two it is essential that no two transactions are ever assigned exactly the same timestamp. As we have seen, Lamport’s algorithm guarantees uniqueness (effectively by using process numbers to break ties).

The idea behind the algorithm is that when one process is about to block

waiting for a resource that another process is using, a check is made to see which has a larger timestamp (i.e., is younger). We can then allow the wait only if the waiting process has a lower timestamp (is older) than the process waited for. In this manner, following any chain of waiting processes, the timestamps always increase, so cycles are impossible. Alternatively, we can allow processes to wait only if the waiting process has a higher timestamp (is younger) than the process waited for, in which case the timestamps decrease along the chain.

Although both methods prevent deadlocks, it is wiser to give priority to older processes. They have run longer, so the system has a larger investment in them, and they are likely to hold more resources. Also, a young process that is killed off will eventually age until it is the oldest one in the system, so this choice eliminates starvation. As we have pointed out before, killing a transaction is relatively harmless, since by definition it can be restarted safely later.

To make this algorithm clearer, consider the situation of Fig. 3-25. In (a), an old process wants a resource held by a young process. In (b), a young process wants a resource held by an old process. In one case we should allow the process to wait; in the other we should kill it. Suppose that we label (a) *dies* and (b) *wait*. Then we are killing off an old process trying to use a resource held by a young process, which is inefficient. Thus we must label it the other way, as shown in the figure. Under these conditions, the arrows always point in the direction of increasing transaction numbers, making cycles impossible. This algorithm is called **wait-die**.

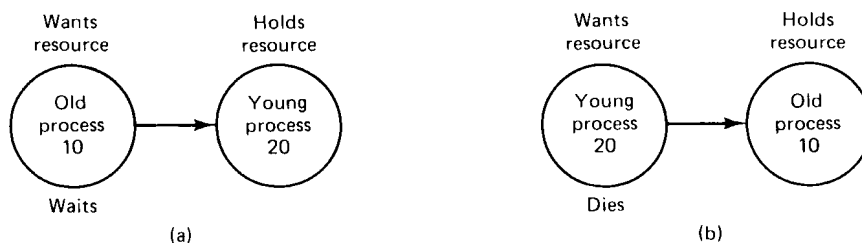


Fig. 3-25. The wait-die deadlock prevention algorithm.

Once we are assuming the existence of transactions, we can do something that had previously been forbidden: take resources away from running processes. In effect we are saying that when a conflict arises, instead of killing the process making the request, we can kill the resource owner. Without transactions, killing a process might have severe consequences, since the process might have modified files, for example. With transactions, these effects will vanish magically when the transaction dies.

Now consider the situation of Fig. 3-26, where we are going to allow

preemption. Given that our system believes in ancestor worship, as we discussed above, we do not want a young whippersnapper preempting a venerable old sage, so Fig. 3-26(a) and not Fig. 3-26(b) is labeled *preempt*. We can now safely label Fig. 3-26(b) *wait*. This algorithm is known as **wound-wait**, because one transaction is supposedly wounded (it is actually killed) and the other waits. It is unlikely that this algorithm will make it to the Nomenclature Hall of Fame.

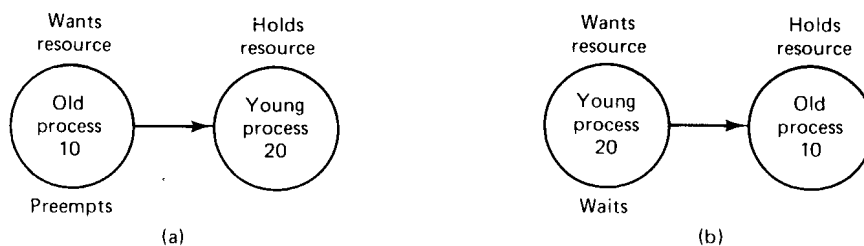


Fig. 3-26. The wound-wait deadlock prevention algorithm.

If an old process wants a resource held by a young one, the old process preempts the young one, whose transaction is then killed, as depicted in Fig. 3-26(a). The young one probably starts up again immediately, and tries to acquire the resource, leading to Fig. 3-26(b), forcing it to wait. Contrast this algorithm with wait-die. There, if an oldtimer wants a resource held by a young squirt, the oldtimer waits politely. However, if the young one wants a resource held by the old one, the young one is killed. It will undoubtedly start up again and be killed again. This cycle may go on many times before the old one releases the resource. Wound-wait does not have this nasty property.

3.6. SUMMARY

This chapter is about synchronization in distributed systems. We started out by giving Lamport's algorithm for synchronizing clocks without reference to external time sources, and later saw how useful this algorithm is. We also saw how physical clocks can be used for synchronization when real time is important.

Next we looked at mutual exclusion in distributed systems and studied three algorithms. The centralized algorithm kept all the information at a single site. The distributed algorithm ran the computation at all sites in parallel. The token ring algorithm passed control around the ring. Each has its strengths and weaknesses.

Many distributed algorithms require a coordinator, so we looked at two ways of electing a coordinator, the bully algorithm and another ring algorithm.

Although all of the foregoing are interesting and important, they are all low-level concepts. Transactions are a high-level concept that makes it easier for the programmer to handle mutual exclusion, locking, fault tolerance, and deadlocks in a distributed system. We looked at the transaction model, how transactions are implemented, and three concurrency control schemes: locking, optimistic concurrency control, and timestamps.

Finally, we revisited the problem of deadlocks and saw some algorithms for detecting and preventing them in distributed systems.

PROBLEMS

1. Add a new message to Fig. 3-2(b) that is concurrent with message *A*, that is, it neither happens before *A* nor happens after *A*.
2. Name at least three sources of delay that can be introduced between WWV broadcasting the time and the processors in a distributed system setting their internal clocks.
3. Consider the behavior of two machines in a distributed system. Both have clocks that are supposed to tick 1000 times per millisecond. One of them actually does, but the other ticks only 990 times per millisecond. If UTC updates come in once a minute, what is the maximum clock skew that will occur?
4. In the approach to cache consistency using leases, is it really essential that the clocks are synchronized? If not, what is it that is required?
5. In the centralized approach to mutual exclusion (Fig. 3-8), upon receiving a message from a process releasing its exclusive access to the critical region it was using, the coordinator normally grants permission to the first process on the queue. Give another possible algorithm for the coordinator.
6. Consider Fig. 3-8 again. Suppose that the coordinator crashes. Does this always bring the system down? If not, under what circumstances does this happen? Is there any way to avoid the problem and make the system able to tolerate coordinator crashes?
7. Ricart and Agrawala's algorithm has the problem that if a process has crashed and does not reply to a request from another process to enter a critical region, the lack of response will be interpreted as denial of permission. We suggested that all requests be answered immediately, to make it easy to detect crashed processes. Are there any circumstances where even this method is insufficient? Discuss.

8. A distributed system may have multiple, independent critical regions. Imagine that process 0 wants to enter critical region *A* and process 1 wants to enter critical region *B*. Can Ricart and Agrawala's algorithm lead to deadlocks? Explain your answer.
9. In Fig. 3-12 a small optimization is possible. What is it?
10. Suppose that two processes detect the demise of the coordinator simultaneously and both decide to hold an election using the bully algorithm. What happens?
11. In Fig. 3-13 we have two *ELECTION* messages circulating simultaneously. While it does no harm to have two of them, it would be more elegant if one could be killed off. Devise an algorithm for doing this without affecting the operation of the basic election algorithm.
12. In Fig. 3-14 we saw a way to update an inventory list atomically using magnetic tape. Since a tape can easily be simulated on disk (as a file), why do you think this method is not used any more?
13. For some ultrasensitive applications it is conceivable that stable storage implemented with two disks is not reliable enough. Can the idea be extended to three disks? If so, how would it work? If not, why not?
14. In Fig. 3-17(d) three schedules are shown, two legal and one illegal. For the same transactions, give a complete list of all values that *x* might have at the end, and state which are legal and which are illegal.
15. When a private workspace is used to implement transactions, it may happen that a large number of file indices must be copied back to the parent's workspace. How can this be done without introducing race conditions?
16. In the writeahead log, both the old and new values are stored in the log entries. Is it not adequate just to store the new value? What good is the old one?
17. In Fig. 3-20, at what instant is the point-of-no-return reached? That is, when is the atomic commit actually performed?
18. Give the full algorithm for whether an attempt to lock a file should succeed or fail. Consider both read and write locks, and the possibility that the file was unlocked, read locked, or write locked.
19. Systems that use locking for concurrency control usually distinguish read locks from write locks. What should happen if a process has already acquired a read lock and now wants to change it into a write lock? What about changing a write lock into a read lock?

20. Is optimistic concurrency control more or less restrictive than using time-stamps? Why?
21. Does using timestamping for concurrency control ensure serializability? Discuss.
22. We have repeatedly said that when a transaction is aborted, the world is restored to its previous state, as though the transaction had never happened. We lied. Give an example where resetting the world is impossible.
23. The centralized deadlock detection algorithm described in the text initially gave a false deadlock, but was later patched up using global time. Suppose that it has been decided not to maintain global time (too expensive). Devise an alternative way to fix the bug in the algorithm.
24. A process with transaction timestamp 50 needs a resource held by a process with transaction timestamp 100. What happens in:
 - (a) Wait-die?
 - (b) Wound-wait?

4

Processes and Processors in Distributed Systems

In the preceding two chapters, we have looked at two related topics, communication and synchronization in distributed systems. In this chapter we will switch to a different subject: processes. Although processes are also an important concept in uniprocessor systems, in this chapter we will emphasize aspects of process management that are usually not studied in the context of classical operating systems. In particular, we will look at how the existence of multiple processors is dealt with.

In many distributed systems, it is possible to have multiple threads of control within a process. This ability provides some important advantages, but also introduces various problems. We will study these issues first. Then we come to the subject of how the processors and processes are organized and see that several different models are possible. Then we will look at processor allocation and scheduling in distributed systems. Finally, we consider two special kinds of distributed systems, fault-tolerant systems and real-time systems.

4.1. THREADS

In most traditional operating systems, each process has an address space and a single thread of control. In fact, that is almost the definition of a process. Nevertheless, there are frequently situations in which it is desirable to have

multiple threads of control sharing one address space but running in quasi-parallel, as though they were separate processes (except for the shared address space). In this section we will discuss these situations and their implications.

4.1.1. Introduction to Threads

Consider, for example, a file server that occasionally has to block waiting for the disk. If the server had multiple threads of control, a second thread could run while the first one was sleeping. The net result would be a higher throughput and better performance. It is not possible to achieve this goal by creating two independent server processes because they must share a common buffer cache, which requires them to be in the same address space. Thus a new mechanism is needed, one that historically was not found in single-processor operating systems.

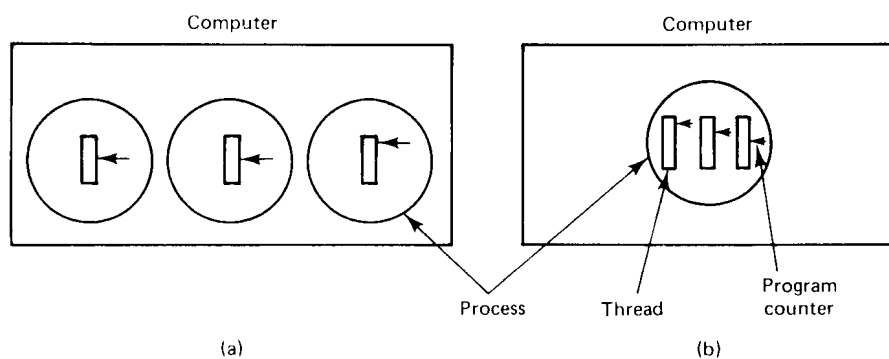


Fig. 4-1. (a) Three processes with one thread each. (b) One process with three threads.

In Fig. 4-1(a) we see a machine with three processes. Each process has its own program counter, its own stack, its own register set, and its own address space. The processes have nothing to do with each other, except that they may be able to communicate through the system's interprocess communication primitives, such as semaphores, monitors, or messages. In Fig. 4-1(b) we see another machine, with one process. Only this process contains multiple threads of control, usually just called **threads**, or sometimes **lightweight processes**. In many respects, threads are like little mini-processes. Each thread runs strictly sequentially and has its own program counter and stack to keep track of where it is. Threads share the CPU just as processes do: first one thread runs, then another does (timesharing). Only on a multiprocessor do they actually run in parallel. Threads can create child threads and can block waiting for system calls to complete, just like regular processes. While one thread is blocked, another thread in

the same process can run, in exactly the same way that when one process blocks, another process in the same machine can run. The analogy: thread is to process as process is to machine, holds in many ways.

Different threads in a process are not quite as independent as different processes, however. All threads have exactly the same address space, which means that they also share the same global variables. Since every thread can access every virtual address, one thread can read, write, or even completely wipe out another thread's stack. There is no protection between threads because (1) it is impossible, and (2) it should not be necessary. Unlike different processes, which may be from different users and which may be hostile to one another, a process is always owned by a single user, who has presumably created multiple threads so that they can cooperate, not fight. In addition to sharing an address space, all the threads share the same set of open files, child processes, timers, and signals, etc. as shown in Fig. 4-2. Thus the organization of Fig. 4-1(a) would be used when the three processes are essentially unrelated, whereas Fig. 4-1(b) would be appropriate when the three threads are actually part of the same job and are actively and closely cooperating with each other.

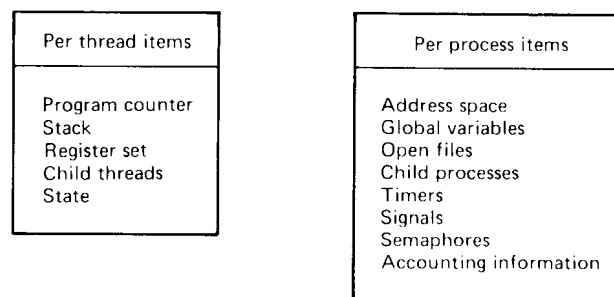


Fig. 4-2. Per thread and per process concepts.

Like traditional processes (i.e., processes with only one thread), threads can be in any one of several states: running, blocked, ready, or terminated. A running thread currently has the CPU and is active. A blocked thread is waiting for another thread to unblock it (e.g., on a semaphore). A ready thread is scheduled to run, and will as soon as its turn comes up. Finally, a terminated thread is one that has exited, but which has not yet been collected by its parent (in UNIX terms, the parent thread has not yet done a WAIT).

4.1.2. Thread Usage

Threads were invented to allow parallelism to be combined with sequential execution and blocking system calls. Consider our file server example again. One possible organization is shown in Fig. 4-3(a). Here one thread, the

dispatcher, reads incoming requests for work from the system mailbox. After examining the request, it chooses an idle (i.e., blocked) **worker thread** and hands it the request, possibly by writing a pointer to the message into a special word associated with each thread. The dispatcher then wakes up the sleeping worker (e.g., by doing an UP on the semaphore on which it is sleeping).

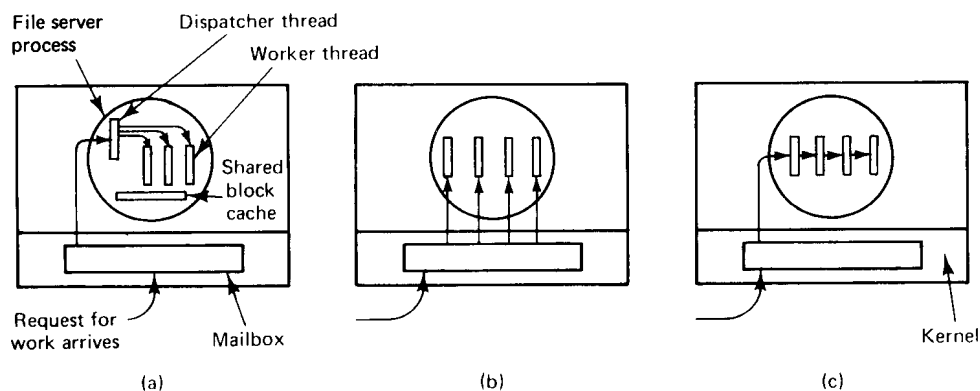


Fig. 4-3. Three organizations of threads in a process. (a) Dispatcher/worker model. (b) Team model. (c) Pipeline model.

When the worker wakes up, it checks to see if the request can be satisfied from the shared block cache, to which all threads have access. If not, it sends a message to the disk to get the needed block (assuming it is a READ) and goes to sleep awaiting completion of the disk operation. The scheduler will now be invoked and another thread will be started, possibly the dispatcher, in order to acquire more work, or possibly another worker that is now ready to run.

Consider how the file server could be written in the absence of threads. One possibility is to have it operate as a single thread. The main loop of the file server gets a request, examines it, and carries it out to completion before getting the next one. While waiting for the disk, the server is idle and does not process any other requests. If the file server is running on a dedicated machine, as is commonly the case, the CPU is simply idle while the file server is waiting for the disk. The net result is that many fewer requests/sec can be processed. Thus threads gain considerable performance, but each thread is programmed sequentially, in the usual way.

So far we have seen two possible designs: a multithreaded file server and a single-threaded file server. Suppose that threads are not available but the system designers find the performance loss due to single threading unacceptable. A third possibility is to run the server as a big finite-state machine. When a request comes in, the one and only thread examines it. If it can be satisfied from the cache, fine, but if not, a message must be sent to the disk.

However, instead of blocking, it records the state of the current request in a table and then goes and gets the next message. The next message may either be a request for new work or a reply from the disk about a previous operation. If it is new work, that work is started. If it is a reply from the disk, the relevant information is fetched from the table and the reply processed. Since it is not permitted to send a message and block waiting for a reply here, RPC cannot be used. The primitives must be nonblocking calls to *send* and *receive*.

In this design, the “sequential process” model that we had in the first two cases is lost. The state of the computation must be explicitly saved and restored in the table for every message sent and received. In effect, we are simulating the threads and their stacks the hard way. The process is being operated as a finite-state machine that gets an event and then reacts to it, depending on what is in it.

It should now be clear what threads have to offer. They make it possible to retain the idea of sequential processes that make blocking system calls (e.g., RPC to talk to the disk) and still achieve parallelism. Blocking system calls make programming easier and parallelism improves performance. The single-threaded server retains the ease of blocking system calls, but gives up performance. The finite-state machine approach achieves high performance through parallelism, but uses nonblocking calls and thus is hard to program. These models are summarized in Fig. 4-4.

Model	Characteristics
Threads	Parallelism, blocking system calls
Single-thread process	No parallelism, blocking system calls
Finite-state machine	Parallelism, nonblocking system calls

Fig. 4-4. Three ways to construct a server.

The dispatcher structure of Fig. 4-3(a) is not the only way to organize a multithreaded process. The **team** model of Fig. 4-3(b) is also a possibility. Here all the threads are equals, and each gets and processes its own requests. There is no dispatcher. Sometimes work comes in that a thread cannot handle, especially if each thread is specialized to handle a particular kind of work. In this case, a job queue can be maintained, with pending work kept in the job queue. With this organization, a thread should check the job queue before looking in the system mailbox.

Threads can also be organized in the **pipeline** model of Fig. 4-3(c). In this model, the first thread generates some data and passes them on to the next thread for processing. The data continue from thread to thread, with processing going on at each step. Although this is not appropriate for file servers, for other

problems, such as the producer-consumer, it may be a good choice. Pipelining is widely used in many areas of computer systems, from the internal structure of RISC CPUs to UNIX command lines.

Threads are frequently also useful for clients. For example, if a client wants a file to be replicated on multiple servers, it can have one thread talk to each server. Another use for client threads is to handle signals, such as interrupts from the keyboard (DEL or BREAK). Instead of letting the signal interrupt the process, one thread is dedicated full time to waiting for signals. Normally, it is blocked, but when a signal comes in, it wakes up and processes the signal. Thus using threads can eliminate the need for user-level interrupts.

Another argument for threads has nothing to do with RPC or communication. Some applications are easier to program using parallel processes, the producer-consumer problem for example. Whether the producer and consumer actually run in parallel is secondary. They are programmed that way because it makes the software design simpler. Since they must share a common buffer, having them in separate processes will not do. Threads fit the bill exactly here.

Finally, although we are not explicitly discussing the subject here, in a multiprocessor system, it is actually possible for the threads in a single address space to run in parallel, on different CPUs. This is, in fact, one of the major ways in which sharing is done on such systems. On the other hand, a properly designed program that uses threads should work equally well on a single CPU that timeshares the threads or on a true multiprocessor, so the software issues are pretty much the same either way.

4.1.3. Design Issues for Threads Packages

A set of primitives (e.g., library calls) available to the user relating to threads is called a **threads package**. In this section we will consider some of the issues concerned with the architecture and functionality of threads packages. In the next section we will consider how threads packages can be implemented.

The first issue we will look at is thread management. Two alternatives are possible here, static threads and dynamic threads. With a static design, the choice of how many threads there will be is made when the program is written or when it is compiled. Each thread is allocated a fixed stack. This approach is simple, but inflexible.

A more general approach is to allow threads to be created and destroyed on-the-fly during execution. The thread creation call usually specifies the thread's main program (as a pointer to a procedure) and a stack size, and may specify other parameters as well, for example, a scheduling priority. The call usually returns a thread identifier to be used in subsequent calls involving the thread. In this model, a process starts out with one (implicit) thread, but can create one or more threads as needed, and these can exit when finished.

Threads can be terminated in one of two ways. A thread can exit voluntarily when it finishes its job, or it can be killed from outside. In this respect, threads are like processes. In many situations, such as the file servers of Fig. 4-3, the threads are created immediately after the process starts up and are never killed.

Since threads share a common memory, they can, and usually do, use it for holding data that are shared among multiple threads, such as the buffers in a producer-consumer system. Access to shared data is usually programmed using critical regions, to prevent multiple threads from trying to access the same data at the same time. Critical regions are most easily implemented using semaphores, monitors, and similar constructions. One technique that is commonly used in threads packages is the **mutex**, which is a kind of watered-down semaphore. A mutex is always in one of two states, unlocked or locked. Two operations are defined on mutexes. The first one, **LOCK**, attempts to lock the mutex. If the mutex is unlocked, the **LOCK** succeeds and the mutex becomes locked in a single atomic action. If two threads try to lock the same mutex at exactly the same instant, an event that is possible only on a multiprocessor, on which different threads run on different CPUs, one of them wins and the other loses. A thread that attempts to lock a mutex that is already locked is blocked.

The **UNLOCK** operation unlocks a mutex. If one or more threads are waiting on the mutex, exactly one of them is released. The rest continue to wait.

Another operation that is sometimes provided is **TRYLOCK**, which attempts to lock a mutex. If the mutex is unlocked, **TRYLOCK** returns a status code indicating success. If, however, the mutex is locked, **TRYLOCK** does not block the thread. Instead, it returns a status code indicating failure.

Mutexes are like binary semaphores (i.e., semaphores that may only have the values 0 or 1). They are not like counting semaphores. Limiting them in this way makes them easier to implement.

Another synchronization feature that is sometimes available in threads packages is the **condition variable**, which is similar to the condition variable used for synchronization in monitors. Each condition variable is normally associated with a mutex at the time it is created. The difference between mutexes and condition variables is that mutexes are used for short-term locking, mostly for guarding the entry to critical regions. Condition variables are used for long-term waiting until a resource becomes available.

The following situation occurs all the time. A thread locks a mutex to gain entry to a critical region. Once inside the critical region, it examines system tables and discovers that some resource it needs is busy. If it simply locks a second mutex (associated with the resource), the outer mutex will remain locked and the thread holding the resource will not be able to enter the critical region to free it. Deadlock results. Unlocking the outer mutex lets other threads into the critical region, causing chaos, so this solution is not acceptable.

One solution is to use condition variables to acquire the resource, as shown

in Fig. 4-5(a). Here, waiting on the condition variable is defined to perform the wait and unlock the mutex atomically. Later, when the thread holding the resource frees it, as shown in Fig. 4-5(b), it calls *wakeup*, which is defined to wakeup either exactly one thread or all the threads waiting on the specified condition variable. The use of WHILE instead of IF in Fig. 4-5(a) guards against the case that the thread is awakened but that someone else seizes the resource before the thread runs.

<pre>lock mutex; check data structures; while(resource busy) wait(condition variable); mark resource as busy; unlock mutex;</pre>	<pre>lock mutex; mark resource as free; unlock mutex; wakeup(condition variable);</pre>
(a)	(b)

Fig. 4-5. Use of mutexes and condition variables.

The need for the ability to wake up all the threads, rather than just one, is demonstrated in the reader-writer problem. When a writer finishes, it may choose to wake up pending writers or pending readers. If it chooses readers, it should wake them all up, not just one. Providing primitives for waking up exactly one thread and for waking up all the threads provides the needed flexibility.

The code of a thread normally consists of multiple procedures, just like a process. These may have local variables, global variables, and procedure parameters. Local variables and parameters do not cause any trouble, but variables that are global to a thread but not global to the entire program do.

As an example, consider the *errno* variable maintained by UNIX. When a process (or a thread) makes a system call that fails, the error code is put into *errno*. In Fig. 4-6, thread 1 executes the system call ACCESS to find out if it has permission to access a certain file. The operating system returns the answer in the global variable *errno*. After control has returned to thread 1, but before it has a chance to read *errno*, the scheduler decides that thread 1 has had enough CPU time for the moment and decides to switch to thread 2. Thread 2 executes an OPEN call that fails, which causes *errno* to be overwritten and thread 1's access code to be lost forever. When thread 1 starts up later, it will read the wrong value and behave incorrectly.

Various solutions to this problem are possible. One is to prohibit global variables altogether. However worthy this ideal may be, it conflicts with much existing software, such as UNIX. Another is to assign each thread its own private global variables, as shown in Fig. 4-7. In this way, each thread has its own private copy of *errno* and other global variables, so conflicts are avoided. In effect, this decision creates a new scoping level, variables visible to all the

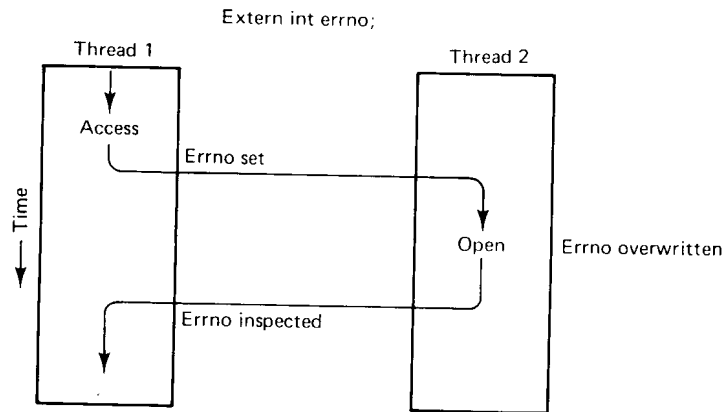


Fig. 4-6. Conflicts between threads over the use of a global variable.

procedures of a thread, in addition to the existing scoping levels of variables visible only to one procedure and variables visible everywhere in the program.

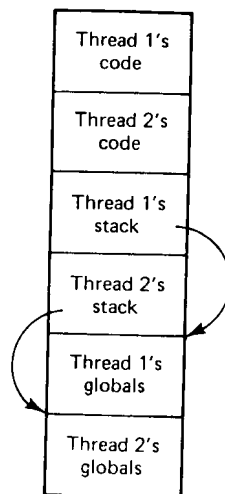


Fig. 4-7. Threads can have private global variables.

Accessing the private global variables is a bit tricky, however, since most programming languages have a way of expressing local variables and global variables, but not intermediate forms. It is possible to allocate a chunk of memory for the globals and pass it to each procedure in the thread, as an extra parameter. While hardly an elegant solution, it works.

Alternatively, new library procedures can be introduced to create, set, and read these thread-wide global variables. The first call might look like this:

```
create_global("bufptr");
```

It allocates storage for a pointer called *bufptr* on the heap or in a special storage area reserved for the calling thread. No matter where the storage is allocated, only the calling thread has access to the global variable. If another thread creates a global variable with the same name, it gets a different storage location that does not conflict with the existing one.

Two calls are needed to access global variables: one for writing them and the other for reading them. For writing, something like

```
set_global("bufptr", &buf);
```

will do. It stores the value of a pointer in the storage location previously created by the call to *create_global*. To read a global variable, the call might look like

```
bufptr = read_global("bufptr");
```

This call returns the address stored in the global variable, so the data value can be accessed.

Our last design issue relating to threads is scheduling. Threads can be scheduled using various scheduling algorithms, including priority, round robin, and others. Threads packages often provide calls to give the user the ability to specify the scheduling algorithm and set the priorities, if any.

4.1.4. Implementing a Threads Package

There are two main ways to implement a threads package: in user space and in the kernel. The choice is moderately controversial, and a hybrid implementation is also possible. In this section we will describe these methods, along with their advantages and disadvantages.

Implementing Threads in User Space

The first method is to put the threads package entirely in user space. The kernel knows nothing about them. As far as the kernel is concerned, it is managing ordinary, single-threaded processes. The first, and most obvious, advantage is that a user-level threads package can be implemented on an operating system that does not support threads. For example, UNIX originally did not support threads, but various user-space threads packages were written for it.

All of these implementations have the same general structure, which is illustrated in Fig. 4-8(a). The threads run on top of a runtime system, which is a

collection of procedures that manage threads. When a thread executes a system call, goes to sleep, performs an operation on a semaphore or mutex, or otherwise does something that may cause it to be suspended, it calls a runtime system procedure. This procedure checks to see if the thread must be suspended. If so, it stores the thread's registers (i.e., its own) in a table, looks for an unblocked thread to run, and reloads the machine registers with the new thread's saved values. As soon as the stack pointer and program counter have been switched, the new thread comes to life again automatically. If the machine has an instruction to store all the registers and another one to load them all, the entire thread switch can be done in a handful of instructions. Doing thread switching like this is at least an order of magnitude faster than trapping to the kernel, and is a strong argument in favor of user-level threads packages.

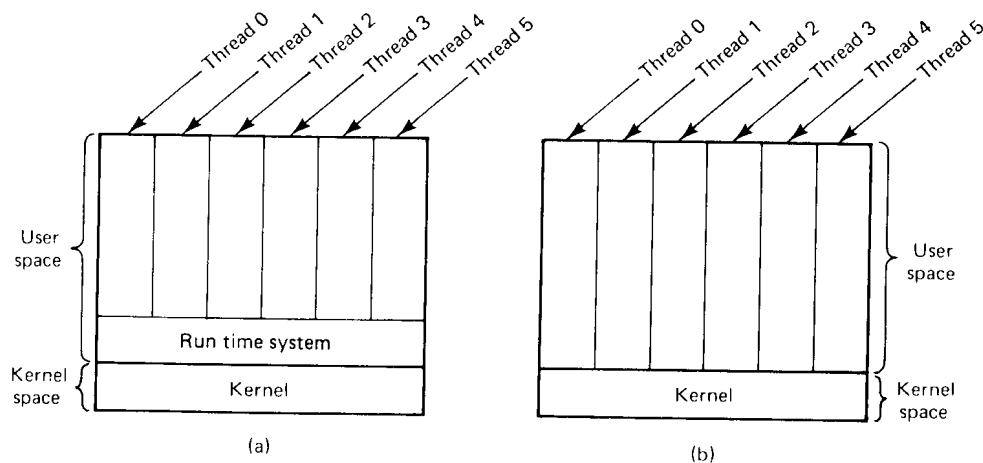


Fig. 4-8. (a) A user-level threads package. (b) A threads package managed by the kernel.

User-level threads also have other advantages. They allow each process to have its own customized scheduling algorithm. For some applications, for example, those with a garbage collector thread, not having to worry about a thread being stopped at an inconvenient moment is a plus. They also scale better, since kernel threads invariably require some table space and stack space in the kernel, which can be a problem if there are a very large number of threads.

Despite their better performance, user-level threads packages have some major problems. First among these is the problem of how blocking system calls are implemented. Suppose that a thread reads from an empty pipe or does something else that will block. Letting the thread actually make the system call is unacceptable, since this will stop all the threads. One of the main goals of

having threads in the first place was to allow each one to use blocking calls, but to prevent one blocked thread from affecting the others. With blocking system calls, this goal cannot be achieved.

The system calls could all be changed to be nonblocking (e.g., a read on a empty pipe could just fail), but requiring changes to the operating system is unattractive. Besides, one of the arguments for user-level threads was precisely that they could run with *existing* operating systems. In addition, changing the semantics of READ will require changes to many user programs.

Another alternative is possible in the event that it is possible to tell in advance if a call will block. In some versions of UNIX, a call SELECT exists, which allows the caller to tell whether a pipe is empty, and so on. When this call is present, the library procedure *read* can be replaced with a new one that first does a SELECT call and then only does the READ call if it is safe (i.e., will not block). If the READ call will block, the call is not made. Instead, another thread is run. The next time the runtime system gets control, it can check again to see if the READ is now safe. This approach requires rewriting parts of the system call library, is inefficient and inelegant, but there is little choice. The code placed around the system call to do the checking is called a **jacket**.

Somewhat analogous to the problem of blocking system calls is the problem of page faults. If a thread causes a page fault, the kernel, not even knowing about the existence of threads, naturally blocks the entire process until the needed page has been fetched, even though other threads might be runnable.

Another problem with user-level thread packages is that if a thread starts running, no other thread in that process will ever run unless the first thread voluntarily gives up the CPU. Within a single process, there are no clock interrupts, making round-robin scheduling impossible. Unless a thread enters the runtime system of its own free will, the scheduler will never get a chance.

An area in which the absence of clock interrupts is crucial is synchronization. It is common in distributed applications for one thread to initiate an activity to which another thread must respond and then just sit in a tight loop testing whether the response has happened. This situation is called a **spin lock** or **busy waiting**. This approach is especially attractive when the response is expected quickly and the cost of using semaphores is high. If threads are rescheduled automatically every few milliseconds based on clock interrupts, this approach works fine. However, if threads run until they block, this approach is a recipe for deadlock.

One possible solution to the problem of threads running forever is to have the runtime system request a clock signal (interrupt) once a second to give it control, but this too is crude and messy to program. Periodic clock interrupts at a higher frequency are not always possible, and even if they are, the total overhead may be substantial. Furthermore, a thread might also need a clock interrupt, interfering with the runtime system's use of the clock.